

[illegible]

[illegible]

[illegible]

```
DecisionType shriji_take_decision(
    int confidence,
    int risk,
    ShrijiErrorCode code
)
{
    (void) confidence;
    (void) risk;

    /* SAFE â\206\222 always auto fix */
    if (is_safe_error(code))
        return DECISION_AUTO_FIX;

    /* AMBIGUOUS â\206\222 ask user */
    if (is_ambiguous_error(code))
        return DECISION_EDIT;

    /* REST â\206\222 reject */
    return DECISION_REJECT;
}
```

```
#include "../include/runtime.h"
#include "../include/state.h"
#include "../include/value.h"
```

[illegible]

```
void runtime_init(ShrijiRuntime *rt)
{
    if (!rt) return;

    current_runtime = rt;

    state_init(&rt->state);

    rt->break_flag = 0;
    rt->continue_flag = 0;
    rt->loop_depth = 0;

    rt->return_flag = 0;
    rt->return_value = value_null();

    rt->call_depth = 0;

    rt->error_flag = 0;

    rt->printed = 0;    /* 0\237\214, NEW */
    rt->executed = 0;
}
```

[illegible]

[illegible]

[illegible]

[illegible]

```

ASTNode *node = parse_program(line);

/* DO NOT RESET error_reported */
if (!node || error_reported)
{
    if (rt) {
        rt->error_flag = 1;
    }
    return value_null();
}

/* EXECUTE AST */
Value result = eval(node, env, rt);
return result;
}

Value eval(ASTNode *node, Env *env, ShrijiRuntime *rt)
{
    /* GLOBAL HARD STOP â\200\224 NO DUPLICATE EXECUTION */
    if (error_reported || (rt && rt->error_flag))
        return value_null();

    static int state_initialized = 0;

    if (!state_initialized) {
        state_init(&rt->state);
        state_initialized = 1;
    }

    if (!state_allows_execution(rt) || !node)
        return value_null();

    /* ð\237$ GLOBAL STEP COUNTER */
    if (node->type != AST_WHILE &&
        node->type != AST_BREAK &&
        node->type != AST_CONTINUE &&
        node->type != AST_BLOCK &&
        node->type != AST_PROGRAM) {
        rt->state.steps_used++;
    }

    if (rt->state.steps_used > rt->state.max_steps) {
        shriji_error(
            E_PARSE_02,
            "runtime",
            "execution limit exceeded",
            "possible infinite loop detected"
        );
        rt->state.safety = STATE_CRITICAL;
        return value_null();
    }

    switch (node->type) {

case AST_BREAK:
    if (rt->loop_depth <= 0) {
        shriji_error(
            E_RUNTIME_01,
            "break",
            "loop ke bahar rukja allowed nahi hai",
            "rukja sirf while loop ke andar use karein"
        );
        return value_null();
    }

```

[illegible]

[illegible]

[illegible]

```

if (!fp) {
    value_free(&pathv);

    shriji_error(
        E_PARSE_02,
        "padho",
        "file not found",
        "check path or create file first"
    );

    return value_string("");
}

fseek(fp, 0, SEEK_END);
long sz = ftell(fp);
fseek(fp, 0, SEEK_SET);

if (sz < 0) sz = 0;
if (sz > 1024 * 1024) sz = 1024 * 1024; /* 1MB limit vl */

char *buf = (char *)malloc((size_t)sz + 1);
if (!buf) {
    fclose(fp);
    value_free(&pathv);
    return value_null();
}

size_t nread = fread(buf, 1, (size_t)sz, fp);
buf[nread] = '\0';

fclose(fp);
value_free(&pathv);

Value out = value_string(buf);
free(buf);
return out;
}

/* WRITE (overwrite) */
if (strcmp(node->function_name, "likho") == 0) {

    if (node->arg_count != 2) {
        shriji_error(
            E_PARSE_02,
            "likho",
            "argument count mismatch",
            "use: likho(\"file.txt\", \"data\")"
        );
        return value_null();
    }

    Value pathv = eval(node->args[0], env, rt);
    Value datav = eval(node->args[1], env, rt);

    if (pathv.type != VAL_STRING || !pathv.string) {
        value_free(&pathv);
        value_free(&datav);
        shriji_error(
            E_PARSE_02,
            "likho",
            "file path must be string",
            "use: likho(\"file.txt\", \"data\")"
        );
        return value_null();
    }

    const char *path = strip_quotes(pathv.string);

    const char *data = "";
    char tmp[128];

```

```

tmp[0] = '\\0';

char *tmp_data = NULL;

if (datav.type == VAL_STRING && datav.string) {
    tmp_data = unescape_string(strip_quotes(datav.string));
    data = tmp_data ? tmp_data : "";
} else if (datav.type == VAL_NUMBER) {
    snprintf(tmp, sizeof(tmp), "%g", datav.number);
    data = tmp;
}

FILE *fp = fopen(path, "wb");
if (!fp) {
    if (tmp_data) free(tmp_data);
    value_free(&pathv);
    value_free(&datav);
    return value_number(0);
}

fwrite(data, 1, strlen(data), fp);
fclose(fp);

if (tmp_data) free(tmp_data);

value_free(&pathv);
value_free(&datav);

return value_number(1);
}

/* APPEND */
if (strcmp(node->function_name, "jodo") == 0) {

    if (node->arg_count != 2) {
        shriji_error(
            E_PARSE_02,
            "jodo",
            "argument count mismatch",
            "use: jodo(\"file.txt\", \"data\")"
        );
        return value_null();
    }

    Value pathv = eval(node->args[0], env, rt);
    Value datav = eval(node->args[1], env, rt);

    if (pathv.type != VAL_STRING || !pathv.string) {
        value_free(&pathv);
        value_free(&datav);
        shriji_error(
            E_PARSE_02,
            "jodo",
            "file path must be string",
            "use: jodo(\"file.txt\", \"data\")"
        );
        return value_null();
    }

    const char *path = strip_quotes(pathv.string);

    const char *data = "";
    char tmp[128];
    tmp[0] = '\\0';

    char *tmp_data = NULL;

    if (datav.type == VAL_STRING && datav.string) {
        tmp_data = unescape_string(strip_quotes(datav.string));

```

[illegible]

[illegible]

[illegible]

[illegible]


```

    if (iv.type != VAL_STRING || !iv.string) goto cleanup;

    /* update if exists */
    for (int i = 0; i < tv.dict_count; i++) {
        Value k = tv.dict_keys[i];
        if (k.type == VAL_STRING && strcmp(k.string, iv.string) == 0) {

            value_free(&tv.dict_values[i]);
            tv.dict_values[i] = value_copy(vv);

            env_set(env, var_name, tv);
            value_free(&iv);
            return value_copy(vv);
        }
    }

    /* append new key */
    tv.dict_keys = realloc(tv.dict_keys,
                          sizeof(Value) * (tv.dict_count + 1));
    tv.dict_values = realloc(tv.dict_values,
                             sizeof(Value) * (tv.dict_count + 1));

    tv.dict_keys[tv.dict_count] = value_copy(iv);
    tv.dict_values[tv.dict_count] = value_copy(vv);
    tv.dict_count++;

    env_set(env, var_name, tv);
    value_free(&iv);
    return value_copy(vv);
}

cleanup:
    value_free(&tv);
    value_free(&iv);
    value_free(&vv);
    return value_null();
}

case AST_IDENTIFIER: {

    if (rt && rt->error_flag)
        return value_null();

    const char *name = node->name;

    if (env_exists(env, name)) {
        Value v = env_get(env, name);
        return value_copy(v);
    }

    if (strcmp(name, "last") == 0) {
        return smriti_session_get_last();
    }

    const char *mem = smriti_personal_get(name);

    if (mem) {
        Value v;
        v.type = VAL_NUMBER;
        v.number = atof(mem);
        return v;
    }

    /* 0\237\224¥ FINAL ERROR */
    shriji_error(
        E_ASSIGN_01,
        "runtime",
        "Variable define nahi hai",

```

```

        name
    );

    if (rt) rt->error_flag = 1;

    return value_null();
}

case AST_ASSIGNMENT: {

    Value value = eval(node->value, env, rt);

    if (error_reported || rt->error_flag) {
        value_free(&value);
        return value_null();
    }

    /* â\234\205 CORRECT STORE */
    env_set(env, node->name, value);

    event_fire(EVENT_ASSIGNMENT, node->name);
    state_on_success(&rt->state);

    Value result = value_copy(value);

    value_free(&value);

    return result;
}

case AST_UPDATE: {

    if (node->name[0] != '\0') {

        if (!env_exists(env, node->name)) {
            shriji_error(
                E_ASSIGN_01,
                node->name,
                "variable not declared",
                "use mavi x = ... first"
            );
            return value_null();
        }

        Value value = eval(node->value, env, rt);

        /* FIXED: use env_update */
        env_update(env, node->name, value);

        event_fire(EVENT_ASSIGNMENT, node->name);
        state_on_success(&rt->state);

        return value;
    }

    shriji_error(
        E_ASSIGN_01,
        "update",
        "invalid update target",
        "use: x = expr"
    );

    return value_null();
}

case AST_BINARY: {

    /* ð\237\224\222 GLOBAL HARD STOP */

```

```

if (error_reported || rt->error_flag)
    return value_null();

/* LEFT */
Value Lv = eval(node->left, env, rt);

if (error_reported || rt->error_flag) {
    value_free(&Lv);
    return value_null();
}

/* RIGHT */
Value Rv = eval(node->right, env, rt);

if (error_reported || rt->error_flag) {
    value_free(&Lv);
    value_free(&Rv);
    return value_null();
}

char op = node->op[0];

/* ===== TYPE CHECK ===== */
if (Lv.type != Rv.type) {
    value_free(&Lv);
    value_free(&Rv);

    shriji_error(
        E_RUNTIME_TYPE_MISMATCH,
        "binary",
        "type mismatch in expression",
        "same type ke values use karein"
    );
    return value_null();
}

/* ===== LOGICAL ===== */
if (op == '&') {
    int res = value_is_truthy(Lv) && value_is_truthy(Rv);
    value_free(&Lv);
    value_free(&Rv);
    return value_bool(res);
}

if (op == '|') {
    int res = value_is_truthy(Lv) || value_is_truthy(Rv);
    value_free(&Lv);
    value_free(&Rv);
    return value_bool(res);
}

/* ===== NUMBER ===== */
if (Lv.type == VAL_NUMBER) {
    double L = Lv.number;
    double R = Rv.number;

    value_free(&Lv);
    value_free(&Rv);

    if (op == '+') return value_number(L + R);
    if (op == '-') return value_number(L - R);
    if (op == '*') return value_number(L * R);
    if (op == '/') return value_number(safe_div(L, R));
    if (op == '%') return value_number(safe_mod(L, R));

    if (op == '>') return value_bool(L > R);
    if (op == '<') return value_bool(L < R);
    if (op == 'G') return value_bool(L >= R);
}

```

```

        if (op == 'L') return value_bool(L <= R);
        if (op == '=') return value_bool(L == R);
        if (op == '!=') return value_bool(L != R);

        shriji_error(
            E_RUNTIME_01,
            "binary",
            "unknown numeric operator",
            NULL
        );
        return value_null();
    }

    /* ===== UNSUPPORTED ===== */
    value_free(&Lv);
    value_free(&Rv);

    shriji_error(
        E_PARSE_02,
        "binary",
        "ye operation is type ke liye allowed nahi hai",
        "sirf number ya string supported hai"
    );

    rt->state.safety = STATE_CRITICAL;
    return value_null();
}

    case AST_NOT: {
        Value v = eval(node->left, env, rt);

        int truth = value_is_truthy(v);
        value_free(&v);

        return value_bool(!truth);
    }

case AST_IF: {
    Value condv = eval(node->condition, env, rt);

    Value result = value_null();

    if (value_is_truthy(condv)) {
        env_push_scope(env);

        result = eval(node->left, env, rt);

        env_pop_scope(env);
    }
    else if (node->else_block) {
        env_push_scope(env);

        result = eval(node->else_block, env, rt);

        env_pop_scope(env);
    }

    value_free(&condv);
    return result;
}

case AST_WHILE: {

```



```

rt->loop_depth++;

int local_guard = 0;

while (1) {

    Value condv = eval(node->condition, env, rt);

    if (!value_is_truthy(condv)) {
        value_free(&condv);
        break;
    }

    value_free(&condv);

    rt->continue_flag = 0;

    eval(node->body, env, rt);

    if (rt->return_flag) {
        rt->loop_depth--;
        return rt->return_value;
    }

    if (rt->break_flag) {
        rt->break_flag = 0;
        break;
    }

    if (rt->continue_flag) {
        rt->continue_flag = 0;
        continue;
    }

    if (++local_guard > 1000000) {
        shriji_error(
            E_RUNTIME_01,
            "loop",
            "infinite loop detected",
            "use rukja or fix condition"
        );
        rt->state.safety = STATE_CRITICAL;
        break;
    }
}

rt->loop_depth--;
return value_null();
}

case AST_BLOCK: {
    Value last = value_null();

    /* NEW SCOPE START */
    env_push_scope(env);

    for (int i = 0; i < node->stmt_count; i++) {

        last = eval(node->statements[i], env, rt);

        if (rt->return_flag || rt->break_flag || rt->continue_flag)
            break;
    }

    /* SCOPE END */
}

```

```

env_pop_scope(env);

if (rt->return_flag)
    return rt->return_value;

return last;
}

case AST_PROGRAM: {

    Value last = value_null();

    for (int i = 0; i < node->stmt_count; i++) {

        Value temp = eval(node->statements[i], env, rt);

        /* runtime safety */
        if (rt && rt->error_flag) {
            value_free(&temp);
            return value_null();
        }

        /* ALWAYS update last */
        value_free(&last);
        last = value_copy(temp);

        value_free(&temp);

        if (rt->break_flag || rt->continue_flag)
            break;
    }

    if (rt->return_flag)
        return rt->return_value;

    return last;
}

case AST_IMPORT: {

    /* ENABLE ANALYSIS MODE */
    shriji_set_error_mode(ERROR_MODE_COLLECT);

    /* import "file.sri" */
    const char *path = node->name;

    if (!path || !*path) {
        shriji_error(
            E_PARSE_02,
            "import",
            "empty import path",
            "use: import \"file.sri\"");
        shriji_set_error_mode(ERROR_MODE_IMMEDIATE);
        return value_null();
    }

    /* read file */
    FILE *fp = fopen(path, "rb");
    if (!fp) {
        shriji_error(
            E_PARSE_02,
            "import",
            "file not found",
            "check path or create file first");
        shriji_set_error_mode(ERROR_MODE_IMMEDIATE);
        return value_null();
    }
}

```

```

}

fseek(fp, 0, SEEK_END);
long sz = ftell(fp);
fseek(fp, 0, SEEK_SET);

if (sz < 0) sz = 0;
if (sz > 1024 * 1024) sz = 1024 * 1024;

char *buf = (char *)malloc((size_t)sz + 1);
if (!buf) {
    fclose(fp);
    shriji_set_error_mode(ERROR_MODE_IMMEDIATE);
    return value_null();
}

size_t nread = fread(buf, 1, (size_t)sz, fp);
buf[nread] = '\0';
fclose(fp);

/* FULL FILE PARSE */
ASTNode *prog = parse_program(buf);
free(buf);

if (!prog) {
    shriji_error(
        E_PARSE_02,
        "import",
        "parse failed",
        "check full file syntax"
    );

    shriji_print_all_errors();
    shriji_set_error_mode(ERROR_MODE_IMMEDIATE);
    return value_null();
}

/* RUN FULL PROGRAM */
Value result = eval(prog, env, rt);

/* PRINT ALL ERRORS (ONE SHOT) */
shriji_print_all_errors();

/* BACK TO NORMAL MODE */
shriji_set_error_mode(ERROR_MODE_IMMEDIATE);

return result;
}

case AST_COMMAND: {
    CommandType cmd = resolve_command(node->command_name);
    if (cmd == CMD_UNKNOWN) {
        shriji_error(
            E_PARSE_02,
            "command",
            "unknown command",
            "command not recognized"
        );
        return value_null();
    }
}

if (cmd == CMD_BOLO)
{
    if (error_reported)
        return value_null();

    Value v = value_null();

```

```

    if (node->value)
        v = eval(node->value, env, rt);

    if (rt->error_flag) {
        value_free(&v);
        return value_null();
    }

    smriti_session_set_last(v);

    return v;    // Ø\237\224¥ RETURN VALUE, DO NOT PRINT
}

/* AI MODULES â\206\222 ai_router (STEP-13 REAL OUTPUT) */
if (cmd == CMD_SAKHI || cmd == CMD_NIYU ||
    cmd == CMD_MIRA || cmd == CMD_KAVYA) {

    Value v = value_null();
    if (node->value)
        v = eval(node->value, env, rt);

    const char *arg = "";
    char tmp[64] = {0};

    if (v.type == VAL_STRING && v.string)
        arg = strip_quotes(v.string);
    else if (v.type == VAL_NUMBER) {
        snprintf(tmp, sizeof(tmp), "%g", v.number);
        arg = tmp;
    }

    ShrijiBridgePacket pkt;
    pkt.text = arg;
    pkt.lang = SHRIJI_LANG_AUTO;
    pkt.source = SHRIJI_SRC_REPL;

    L3Response r = ai_router_dispatch(&pkt);
    if (r.text && *r.text)

        value_free(&v);
        return value_null();
}

/* DEFAULT COMMAND */
return value_number(execute_command(cmd));
}

default:
    return value_null();
}
}

```

```

=====
FILE: ./src/value_print.c
=====

```

```

#include <stdio.h>
#include "../include/value.h"

```

```

/*
 * ShrijiLang Value Print System
 * -----
 * RULES:
 * - Final result MUST be printed with "OUTPUT:" prefix
 * - No empty printf("") calls
 * - Safe handling for NULL values
 * - Clean and extensible structure
 */

```

```

void value_print(Value v)
{
    // Prefix for UI parsing
    printf("OUTPUT: ");

    switch (v.type)
    {
        case VAL_STRING:
        {
            if (v.string != NULL)
                printf("%s", v.string);
            break;
        }

        case VAL_NUMBER:
        {
            printf("%g", v.number);
            break;
        }

        case VAL_BOOL:
        {
            printf("%s", v.boolean ? "true" : "false");
            break;
        }

        case VAL_LIST:
        {
            printf("[");
            for (int i = 0; i < v.count; i++)
            {
                if (i > 0)
                    printf(", ");

                Value item = v.items[i];

                if (item.type == VAL_NUMBER)
                {
                    printf("%g", item.number);
                }
                else if (item.type == VAL_STRING)
                {
                    if (item.string != NULL)
                        printf("\"%s\"", item.string);
                    else
                        printf("\"\"");
                }
                else if (item.type == VAL_BOOL)
                {
                    printf("%s", item.boolean ? "true" : "false");
                }
                else
                {
                    printf("null");
                }
            }
            printf("]");
            break;
        }

        default:
        {
            printf("null");
            break;
        }
    }
}

// Always end with newline

```

```

    printf("\n");
}

=====
FILE: ./src/normalize_engine.c
=====
#include <string.h>
#include <ctype.h>
#include <stdio.h>
#include <stdio.h>

#include "../include/normalize_engine.h"

#define MAX_WORDS 50
#define MAX_WORD_LEN 64

/* =====
   BASIC UTILS
   ===== */

static void to_lowercase(char *s)
{
    for (int i = 0; s[i]; i++)
        s[i] = tolower(s[i]);
}

static void trim_spaces(char *s)
{
    int i = 0, j = 0;

    while (isspace(s[i])) i++;

    while (s[i])
    {
        if (!(isspace(s[i]) && isspace(s[i+1])))
            s[j++] = s[i];
        i++;
    }

    if (j > 0 && isspace(s[j-1])) j--;
    s[j] = '\0';
}

/* =====
   TYPO DICTIONARY
   ===== */

typedef struct {
    const char *wrong;
    const char *correct;
} TypoRule;

static TypoRule rules[] = {
    {"reeor", "error"},
    {"eror", "error"},
    {"softwere", "software"},
    {"creat", "create"},
    {"keise", "kaise"},
    {"helo", "hello"},
    {"hy", "hi"},
    {"sotwere", "software"},
    {"cteate", "create"},
    {"by", "bye"},
};

static int rule_count = sizeof(rules)/sizeof(TypoRule);

```

```

/* =====
WORD SPLIT
===== */

static void split_words(char *input, char words[][MAX_WORD_LEN], int *count)
{
    char *token = strtok(input, " ");
    int i = 0;

    while (token && i < MAX_WORDS)
    {
        strncpy(words[i], token, MAX_WORD_LEN - 1);
        words[i][MAX_WORD_LEN - 1] = '\0';
        i++;
        token = strtok(NULL, " ");
    }

    *count = i;
}

/* =====
APPLY TYPO RULES
===== */

static void apply_typo_rules(char words[][MAX_WORD_LEN], int count)
{
    for (int i = 0; i < count; i++)
    {
        for (int j = 0; j < rule_count; j++)
        {
            if (strcmp(words[i], rules[j].wrong) == 0)
            {
                strncpy(words[i], rules[j].correct, MAX_WORD_LEN - 1);
            }
        }
    }
}

/* =====
JOIN WORDS BACK
===== */

static void join_words(char *output, char words[][MAX_WORD_LEN], int count)
{
    output[0] = '\0';

    for (int i = 0; i < count; i++)
    {
        strncat(output, words[i], MAX_WORD_LEN - 1);
        if (i < count - 1)
            strncat(output, " ", 1);
    }
}

/* =====
SPECIAL CASE: EXIT NIRMAN
===== */

static void fix_joined_commands(char *input)
{
    if (strstr(input, "exitnirman"))
    {
        strcpy(input, "exit nirman");
    }
}

/* =====
MAIN FUNCTION
===== */

```

```

void normalize_input(char *input)
{
    if (!input || !*input)
        return;

    char temp[512];
    strncpy(temp, input, sizeof(temp)-1);
    temp[sizeof(temp)-1] = '\0';

    to_lowercase(temp);
    trim_spaces(temp);

    fix_joined_commands(temp);

    char words[MAX_WORDS][MAX_WORD_LEN];
    int count = 0;

    split_words(temp, words, &count);
    apply_typo_rules(words, count);

    char result[512];
    join_words(result, words, count);

    /* safe copy back */
    snprintf(input, 512, "%s", result);
}

```

```

=====
FILE: ./src/smriti_personal.c
=====
#include "../include/smriti_personal.h"
#include <string.h>
#include <stdio.h>

#define MAX_KEYS 100

typedef struct {
    char key[32];
    char value[128];
} MemoryItem;

/* =====
INTERNAL MEMORY (RAM)
===== */
static MemoryItem memory[MAX_KEYS];
static int memory_count = 0;

/* =====
FILE PATH
===== */
static const char *SMRITI_FILE = ".shriji/smriti.db";

/* =====
LOAD FROM FILE
===== */
void smriti_personal_load(void)
{
    FILE *fp = fopen(SMRITI_FILE, "r");
    if (!fp) return;

    char line[256];

    while (fgets(line, sizeof(line), fp))
    {
        char *eq = strchr(line, '=');
        if (!eq) continue;
    }
}

```



```

        *eq = '\0';

        char *key = line;
        char *value = eq + 1;

        /* remove newline */
        value[strcspn(value, "\n")] = 0;

        smriti_personal_set(key, value);
    }

    fclose(fp);
}

/* =====
   SAVE TO FILE
   ===== */
void smriti_personal_save(void)
{
    FILE *fp = fopen(SMRITI_FILE, "w");
    if (!fp) return;

    for (int i = 0; i < memory_count; i++)
    {
        fprintf(fp, "%s=%s\n", memory[i].key, memory[i].value);
    }

    fclose(fp);
}

/* =====
   SET (WRITE)
   ===== */
void smriti_personal_set(const char *key, const char *value)
{
    if (!key || !value) return;

    /* overwrite if exists */
    for (int i = 0; i < memory_count; i++)
    {
        if (strcmp(memory[i].key, key) == 0)
        {
            strncpy(memory[i].value, value, sizeof(memory[i].value)-1);
            memory[i].value[sizeof(memory[i].value)-1] = '\0';

            smriti_personal_save();
            return;
        }
    }

    /* new entry */
    if (memory_count < MAX_KEYS)
    {
        strncpy(memory[memory_count].key, key, sizeof(memory[memory_count].key)-1);
        strncpy(memory[memory_count].value, value, sizeof(memory[memory_count].value)-1);

        memory[memory_count].key[sizeof(memory[memory_count].key)-1] = '\0';
        memory[memory_count].value[sizeof(memory[memory_count].value)-1] = '\0';

        memory_count++;

        smriti_personal_save();
    }
}

/* =====
   GET (READ)
   ===== */
const char* smriti_personal_get(const char *key)

```

```

{
    if (!key) return NULL;

    for (int i = 0; i < memory_count; i++)
    {
        if (strcmp(memory[i].key, key) == 0)
        {
            return memory[i].value;
        }
    }

    return NULL;
}

/* =====
   CLEAR (RAM ONLY)
   ===== */
void smriti_personal_clear(void)
{
    memory_count = 0;
}

=====
FILE: ./src/nirman_flow_engine.c
=====
#include <stdio.h>
#include <string.h>
#include "../include/nirman.h"

void nirman_flow_process(char *line)
{
    int intent = nirman_detect_intent(line);

    /* STAGE 0 â\206\222 START */
    if (intent == 2 && NIRMAN_CTX.flow_stage == 0)
    {
        printf("Aap kya banana chahte ho?\n");
        NIRMAN_CTX.flow_stage = 1;
        return;
    }

    /* STAGE 1 â\206\222 GOAL CAPTURE */
    if (NIRMAN_CTX.flow_stage == 1)
    {
        if (strstr(line, "software") || strstr(line, "create"))
        {
            printf("Aap kya banana chahte ho?\n");
            return;
        }

        strncpy(NIRMAN_CTX.goal, line, sizeof(NIRMAN_CTX.goal) - 1);
        printf("[FLOW] Target set: %s\n", NIRMAN_CTX.goal);

        NIRMAN_CTX.flow_stage = 2;
        return;
    }

    /* STAGE 2 â\206\222 ASK TYPE */
    if (NIRMAN_CTX.flow_stage == 2)
    {
        if (strstr(NIRMAN_CTX.goal, "calculator"))
        {
            printf("Aapko kis type ka calculator chahiye?\n");
            NIRMAN_CTX.flow_stage = 3;
            return;
        }
    }
}

```

```

/* STAGE 3 â\206\222 HANDLE TYPE */
if (NIRMAN_CTX.flow_stage == 3)
{
    if (strstr(NIRMAN_CTX.goal, "calculator"))
    {
        if (strstr(line, "normal"))
        {
            printf("[FLOW] Type selected: normal calculator\n");
            printf("Agla step: features define karte hain\n");

            NIRMAN_CTX.flow_stage = 4;
        }
        else if (strstr(line, "scientific"))
        {
            printf("[FLOW] Type selected: scientific calculator\n");
            printf("Agla step: features define karte hain\n");

            NIRMAN_CTX.flow_stage = 4;
        }
        else
        {
            printf("Type clear nahi hai (normal / scientific)\n");
        }
        return;
    }
}

/* STAGE 4 â\206\222 NEXT STEP (placeholder) */
if (NIRMAN_CTX.flow_stage == 4)
{
    printf("[FLOW] Feature planning phase...\n");
    return;
}
}

```

```

=====
FILE: ./src/error_intelligence.c
=====

```

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#ifdef _WIN32
#include <readline/readline.h>
#include <readline/history.h>
#endif

#include "../include/error.h"
#include "../include/pragya_avastha.h"
#include "../include/fix_rules.h"

/* =====
   PREFILL (FOR EDIT MODE)
   ===== */

static char safe_buffer[256];
static char *prefill_text = NULL;

#ifdef _WIN32
static int prefill_hook(void)
{
    if (prefill_text)
    {
        rl_insert_text(prefill_text);
        rl_point = strlen(prefill_text);
        prefill_text = NULL;
    }
}

```

```

        return 0;
    }
#endif

/* =====
   USER INPUT (STRICT)
   ===== */

static char get_user_choice()
{
#ifdef _WIN32
    char buffer[16];

    if (!fgets(buffer, sizeof(buffer), stdin))
        return 'i';

    char c = tolower(buffer[0]);

    if (c == 'e') return 'e';
    if (c == 'i') return 'i';

    return 'i';
#else
    char *line = readline("");

    if (!line || !*line)
    {
        free(line);
        return 'i';
    }

    char c = tolower(line[0]);
    free(line);

    if (c == 'e') return 'e';
    if (c == 'i') return 'i';

    return 'i';
#endif
}

/* =====
   POINTER DISPLAY
   ===== */

static void print_pointer(const char *text, int pos)
{
    if (!text || pos < 0) return;

    printf("%s\n", text);

    for (int i = 0; i < pos; i++)
        printf(" ");

    printf("^\\n\\n");
}

/* =====
   BASIC VALIDATION
   ===== */

static int contains_invalid_token(const char *input)
{
    for (int i = 0; input[i]; i++)
    {
        char c = input[i];

        if (isalnum(c) || isspace(c) || strchr("+-*/()._{}[]=<>!\\"'"", c))
            continue;
    }
}

```

```

        return 1;
    }
    return 0;
}

static int has_unmatched_brackets(const char *input)
{
    int count = 0;

    for (int i = 0; input[i]; i++)
    {
        if (input[i] == '(') count++;
        else if (input[i] == ')') count--;
    }

    return count != 0;
}

static int is_valid_input(const char *input)
{
    if (!input || !*input) return 0;
    if (contains_invalid_token(input)) return 0;
    if (has_unmatched_brackets(input)) return 0;
    return 1;
}

/* =====
SIMPLE OPERATOR CLEAN FIX
===== */

static int filter_operators(const char *input, char *output)
{
    int i, j = 0;
    int last_was_op = 0;

    for (i = 0; input[i]; i++)
    {
        char c = input[i];

        if (strchr("+-*/", c))
        {
            if (last_was_op) continue;

            output[j++] = c;
            last_was_op = 1;
        }
        else
        {
            output[j++] = c;
            last_was_op = 0;
        }
    }

    output[j] = '\0';
    return strcmp(input, output) != 0;
}

/* =====
MAIN ERROR INTELLIGENCE
===== */

void shriji_error_intelligence(
    PragyaAvastha *avastha,
    const ShrijiErrorInfo *err,
    void *niyu_result)
{
    (void)niyu_result;
}

```

```

if (!avastha || !err || !avastha->raw_text)
    return;

/* 0\237\224\222 GLOBAL STOP (avoid duplicate flow) */

if (!avastha->raw_text)
    return;
const char *text = avastha->raw_text;

/* ===== VALIDATION ===== */
if (!is_valid_input(text))
{
    avastha->stop_execution = 1;
    avastha->has_correction = 0;

    // pointer handled by mira layer
    return;
}
/* ===== RULE BASED FIX ===== */

FixRule *rule = shriji_get_rule_for_error(err->code);

if (rule && rule->apply_fix)
{
    char buffer[256];
    snprintf(buffer, sizeof(buffer), "%s", text);

    if (rule->apply_fix(buffer, sizeof(buffer), err))
    {
        if (rule->safety == FIX_SAFE_DETERMINISTIC)
        {
            /* ONLY PREPARE FIX 0\200\224 DO NOT APPLY */

            avastha->has_correction = 1;

            snprintf(avastha->corrected_text,
                sizeof(avastha->corrected_text),
                "%s", buffer);

            return;
        }

        printf("\nSuggestion: %s\n", buffer);
        printf("\nEdit (E) / Ignore (I): ");

        char choice = get_user_choice();

        if (choice == 'e')
        {
            avastha->stop_execution = 1;
            avastha->has_correction = 0;

            printf("\nEdit:\n%s\n", text);

#ifdef _WIN32
            snprintf(safe_buffer, sizeof(safe_buffer), "%s", text);
            prefill_text = safe_buffer;
            rl_startup_hook = prefill_hook;
#endif

            return;
        }

        avastha->stop_execution = 1;
        avastha->has_correction = 0;
        return;
    }
}

```

```

    }
}

/* ===== SIMPLE FILTER FIX ===== */

char filtered[256];

if (filter_operators(text, filtered))
{
    printf("\nSuggestion: %s\n", filtered);
    printf("\nEdit (E) / Ignore (I): ");

    char choice = get_user_choice();

    if (choice == 'e')
    {
        avastha->stop_execution = 1;
        avastha->has_correction = 0;

        printf("\nEdit:\n%s\n", text);

#ifdef _WIN32
        snprintf(safe_buffer, sizeof(safe_buffer), "%s", text);
        prefill_text = safe_buffer;
        rl_startup_hook = prefill_hook;
#endif
        return;
    }

    avastha->stop_execution = 1;
    avastha->has_correction = 0;
    return;
}

/* ===== FALLBACK POINTER ===== */
/* ===== FALLBACK POINTER ===== */

// pointer handled by mira layer

avastha->stop_execution = 1;
avastha->has_correction = 0;

}

```

```

=====
FILE: ./src/example_builder.c
=====
#include <stdio.h>
#include <string.h>

/*
    Build readable example from user input
*/

void shriji_build_example(const char *input, char *out, int size)
{
    int i = 0;
    int j = 0;

    while (input[i] && j < size - 1)
    {
        char c = input[i];

        if (c == '+' || c == '-' || c == '*' || c == '/')
        {
            if (j < size - 3)
            {
                out[j++] = ' ';
            }
        }
    }
}

```


[illegible]

[illegible]

```
=====
FILE: ./src/error.c
=====
```

```
{
    E_RUNTIME_DIV_ZERO,
    "Divide by Zero",
    "0 se divide nahi kar sakte",
    "denominator 0 nahi hona chahiye",
    "10 / 2 try karo"
},
{
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

{
    return last_input;
}

/* =====
   LAST OUTPUT
   ===== */

void smriti_session_set_last_output(const char *text)
{
    if (!text) return;

    strncpy(last_output, text, sizeof(last_output) - 1);
    last_output[sizeof(last_output) - 1] = '\0';
}

const char* smriti_session_get_last_output(void)
{
    return last_output;
}

/* =====
   LAST VALUE (UPGRADED)
   ===== */

void smriti_session_set_last(Value v)
{
    if (has_last)
        value_free(&last_value);

    last_value = value_copy(v);
    has_last = 1;
}

Value smriti_session_get_last(void)
{
    if (!has_last)
        return value_null();

    return value_copy(last_value);
}

/* =====
   CLEAR SESSION
   ===== */

void smriti_session_clear(void)
{
    last_input[0] = '\0';
    last_output[0] = '\0';

    if (has_last)
        value_free(&last_value);

    has_last = 0;
}

```

```

=====
FILE: ./src/event.c
=====
#include <string.h>

#include "../include/event.h"
#include "../include/state.h"
#include "../include/runtime.h"

/* Internal runtime binding */
static ShrijiRuntime *BOUND_RUNTIME = NULL;

```

[illegible]

[illegible]

```
ASTNode *new_update_node(const char *name, ASTNode *val)
{
    ASTNode *node = calloc(1, sizeof(ASTNode));
    if (!node) return NULL;

    node->type = AST_UPDATE;
    safe_copy(node->name, name, 128);
    node->value = val;
    node->chakra_state = CHAKRA_OK;
    return node;
}
```

```
AI_NODE(new_sakhi_node,  AST_SAKHI)
AI_NODE(new_niyu_node,   AST_NIYU)
AI_NODE(new_shiri_node,  AST_SHIRI)
AI_NODE(new_mira_node,   AST_MIRA)
AI_NODE(new_kavya_node,  AST_KAVYA)
```

[illegible]

[illegible]

[illegible]

[illegible]


```

=====
FILE: ./src/fix_interactive.c
=====
#include <stdio.h>
#include <string.h>

#include "../include/fix_interactive.h"
#include "../include/error.h"

/* =====
   MEMORY â\200\224 LAST INPUT TRACKING
   ===== */

static char last_input[512] = {0};

/* =====
   SAFE LINE INPUT
   ===== */

static int read_line(char *buffer, int size)
{
    if (!fgets(buffer, size, stdin))
        return 0;

    buffer[strcspn(buffer, "\n")] = '\0';
    return 1;
}

/* =====
   SAME INPUT GUARD
   ===== */

static int is_same_input(const char *input)
{
    if (strcmp(last_input, input) == 0)
    {
        printf("\n[SYSTEM] Same input detected. Please modify it.\n");
        return 1;
    }

    strncpy(last_input, input, sizeof(last_input) - 1);
    last_input[sizeof(last_input) - 1] = '\0';

    return 0;
}

/* =====
   FIX MENU CORE (SHARED)
   ===== */

static int show_fix_menu(const char *input, char *output, int size)
{
    while (1)
    {
        printf("\nFix Options:\n\n");
        printf("M) Manual edit\n");
        printf("E) Quick edit\n");
        printf("I) Ignore\n");

        printf("\nSelect: ");

        char choice = getchar();
        getchar(); /* consume newline */

        /* =====
           M â\200\224 Manual Edit
           ===== */
    }
}

```

```

    if (choice == 'M' || choice == 'm')
    {
        printf("Enter new expression: ");

        if (!read_line(output, size))
            return 0;

        if (is_same_input(output))
            continue;

        return 1;
    }

    /* =====
       E â\200\224 Quick Edit
       ===== */
    if (choice == 'E' || choice == 'e')
    {
        printf("Edit:\n%s\n> ", input);

        if (!read_line(output, size))
            return 0;

        if (is_same_input(output))
            continue;

        return 1;
    }

    /* =====
       I â\200\224 Ignore
       ===== */
    if (choice == 'I' || choice == 'i')
    {
        return 0;
    }

    printf("Invalid choice. Try again.\n");
}

/* =====
   PUBLIC WRAPPERS
   ===== */

int shriji_interactive_double_operator_fix(
    const char *input,
    char *output,
    int size)
{
    return show_fix_menu(input, output, size);
}

int shriji_interactive_bracket_fix(
    const char *input,
    char *output,
    int size)
{
    return show_fix_menu(input, output, size);
}

int shriji_interactive_operator_fix(
    const char *input,
    char *output,
    int size)
{
    return show_fix_menu(input, output, size);
}

```

[illegible]

```

        while (1)
        {
#ifdef _WIN32
            char *line = readline("Shiri> ");
            if (!line) break;
#else
            char line_local[1024];
            printf("Shiri> ");
            if (!fgets(line_local, sizeof(line_local), stdin))
                break;

            line_local[strcspn(line_local, "\n")] = 0;
            char *line = line_local;
#endif

            if (strcmp(line, "exit") == 0)
            {
#ifdef _WIN32
                free(line);
#endif
                printf("ð\237\214, Exit Shriji ð\237\214,\n");
                break;
            }

            if (strlen(line) == 0)
            {
#ifdef _WIN32
                free(line);
#endif
                continue;
            }

#ifdef _WIN32
            add_history(line);
#endif

            execute_input(line);

#ifdef _WIN32
            free(line);
#endif
        }

/* =====
FILE MODE
===== */
static void run_file_mode(const char *filename)
{
    FILE *fp = fopen(filename, "rb");

    if (!fp)
    {
        printf("File open failed: %s\n", filename);
        return;
    }

    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
    rewind(fp);

    char *buffer = (char *)malloc(size + 1);
    if (!buffer)
    {
        fclose(fp);
        return;
    }
}

```


[illegible]

[illegible]


```

case CMD_SAMAY: {
    /* â\234\205 print system time in 12-hour format (AM/PM) */
    event_fire(EVENT_EXECUTION_SUCCESS, "samay executed");

    time_t now = time(NULL);
    struct tm *t = localtime(&now);

    if (!t) {
        printf("00:00 AM\n");
        return 0;
    }

    int hour = t->tm_hour;
    int minute = t->tm_min;

    const char *ampm = "AM";
    if (hour >= 12) ampm = "PM";

    hour = hour % 12;
    if (hour == 0) hour = 12;

    printf("%02d:%02d %s\n", hour, minute, ampm);
    return 0;
}

case CMD_SAKHI:
case CMD_NIYU:
case CMD_MIRA:
case CMD_KAVYA:
case CMD_SHIRI:
    /* handled inside interpreter because requires args */
    event_fire(EVENT_ERROR, "AI command requires args");
    return -1;
default:
    event_fire(EVENT_ERROR, "command not implemented");
    return -1;
}
}

```

```
=====
```

```
FILE: ./src/atma_lekha.c
```

```
=====
```

```

#include "../include/atma_lekha.h"
#include <stdio.h>
#include <time.h>

```

```
static const char *ATMA_FILE = ".shriji/atma_lekha.log";
```

```

void atma_lekha_init(void)
{
    FILE *fp = fopen(ATMA_FILE, "a");
    if (fp) fclose(fp);
}

```

```

void atma_lekha_note(AtmaEventType type,
                    const char *component,
                    const char *detail)

```

```

{
    FILE *fp = fopen(ATMA_FILE, "a");
    if (!fp) return;

    time_t now = time(NULL);
    struct tm *t = localtime(&now);

    fprintf(fp,
        "[%04d-%02d-%02d %02d:%02d:%02d] | %d | %s | %s\n",
        t->tm_year + 1900,

```

```

        t->tm_mon + 1,
        t->tm_mday,
        t->tm_hour,
        t->tm_min,
        t->tm_sec,
        type,
        component ? component : "unknown",
        detail ? detail : "-"
    );

    fclose(fp);
}

```

```

=====
FILE: ./src/typo_engine.c
=====

```

```

#include <string.h>
#include "../include/typo_engine.h"

```

```

/*
    Simple exit typo detection

    Rule:
    - Length must be 4
    - At least 3 characters match with "exit"
*/

```

```

int shriji_check_typo(
    const char *input,
    char *suggestion,
    int suggestion_size
)
{
    if (!input || !suggestion)
        return 0;

    const char *target = "exit";

    if (strlen(input) != 4)
        return 0;

    int match = 0;

    for (int i = 0; i < 4; i++) {
        if (input[i] == target[i])
            match++;
    }

    if (match >= 3) {
        strncpy(suggestion, target, suggestion_size - 1);
        suggestion[suggestion_size - 1] = '\0';
        return 1;
    }

    return 0;
}

```

```

=====
FILE: ./src/parser.c
=====

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "../include/parser.h"
#include "../include/token.h"
#include "../include/ast.h"

```

[illegible]

```

static void advance(void)
{
    prev_token = current;

    do {
        current = scan_token();
    } while (current.type == TOKEN_NEWLINE);
}

static int expect(TokenType t, ShrijiErrorCode c,
                  const char *ctx, const char *msg, const char *hint)
{
    if (current.type == t) {
        advance();
        return 1;
    }

    shriji_error_at(current, c, ctx, msg, hint);
    return 0;
}

static void skip_separators(void)
{
    while (current.type == TOKEN_NEWLINE ||
           current.type == TOKEN_SEMICOLON)
    {
        advance();
    }
}

/* SEPARATOR CHECK (MISSING FUNCTION FIX)
 *
 */
static int is_separator(TokenType type)
{
    return (type == TOKEN_NEWLINE ||
            type == TOKEN_SEMICOLON);
}

static void unescape_into(char *out, int outcap, const char *in, int inlen)
{
    int oi = 0;

    for (int i = 0; i < inlen && oi < outcap - 1; i++) {
        char c = in[i];

        if (c == '\\') {
            char n = in[i + 1];
            i++;

            switch (n) {

```

[illegible]

[illegible]

[illegible]

```

        return new_continue_node();
    }

/* bolo <expr> */
if (current.type == TOKEN_BOLO) {
    advance();
    skip_separators();

    node = parse_expression();

    if (!node) {
        /* expression layer already handled error */
        return NULL;
    }

    skip_separators();
    return new_command_node("bolo", node);
}

/* AI commands */
if (current.type == TOKEN_SAKHI ||
    current.type == TOKEN_NIYU ||
    current.type == TOKEN_MIRA ||
    current.type == TOKEN_KAVYA ||
    current.type == TOKEN_SHIRI) {

char cmd[32];
int len = current.length;

if (len >= (int)sizeof(cmd))
    len = (int)sizeof(cmd) - 1;

strncpy(cmd, current.start, len);
cmd[len] = '\0';

advance();
skip_separators();

/* â\234\205 OPTIONAL ARG */
ASTNode *arg = NULL;

if (!is_separator(current.type)) {
    arg = parse_value();
}

skip_separators();
return new_command_node(cmd, arg);
}

/* '=' can never start a statement */
if (current.type == TOKEN_EQUAL) {

    char example[64];
    shriji_build_example(current_source, example, sizeof(example));

    if (!error_reported)
    {
        shriji_error_at(
            current,
            E_PARSE_INVALID_TOKEN,
            "statement",
            "Assignment yahin se start nahi ho sakta. Variable aur '=' ek hi line me likho.
",
            example
        );
        error_reported = 1;    // CRITICAL
    }
}

```

[illegible]

[illegible]

[illegible]

[illegible]

```

return NULL;
}

char fname[128];
strncpy(fname, current.start, current.length);
fname[current.length] = 0;
advance(); /* consume function name */

skip_separators();

if (!expect(TOKEN_LEFT_PAREN, E_PARSE_02, "(", "missing (",
            "kaam add(a, b) { ... }"))
    return NULL;

ASTNode **params = NULL;
int param_count = 0;

skip_separators();

if (current.type != TOKEN_RIGHT_PAREN) {
    while (1) {

        if (current.type != TOKEN_IDENTIFIER) {
            if (!error_reported)
            {
                shriji_error_at(current, E_FUNCTION_PARAM_INVALID, "params",
                                "expected parameter name",
                                "kaam add(a, b) { ... }");
                error_reported = 1;    // ð\237\224¥ CRITICAL
            }
            free(params);
            return NULL;
        }

        char p[128];
        strncpy(p, current.start, current.length);
        p[current.length] = 0;
        advance();

        ASTNode **tmp = realloc(params, sizeof(ASTNode*) * (param_count + 1));
        if (!tmp) {
            free(params);
            return NULL;
        }
        params = tmp;

        params[param_count++] = new_identifier_node(p);

        skip_separators();

        if (current.type == TOKEN_COMMA) {
            advance();
            skip_separators();
        }

        if (current.type == TOKEN_RIGHT_PAREN)
            break;
    }
}

if (!expect(TOKEN_RIGHT_PAREN, E_PARSE_BRACKET_MISSING, ")", "missing )",
            "kaam add(a, b) { ... }")) {
    free(params);
    return NULL;
}

```



```

}

skip_separators();

ASTNode *body = parse_block();
if (!body) return NULL;

skip_separators();
return new_function_node(fname, params, param_count, body);
}

static ASTNode *parse_rachna_definition(void)
{
    advance(); // consume 'rachna'
    skip_separators();

    /* name */
    if (current.type != TOKEN_IDENTIFIER) {
        shriji_error_at(current, E_PARSE_INVALID_TOKEN,
            "rachna",
            "name expected",
            "example: rachna add() { }");
        return NULL;
    }

    char name[128];
    strncpy(name, current.start, current.length);
    name[current.length] = '\0';

    advance();
    skip_separators();

    /* '(' */
    if (!expect(TOKEN_LEFT_PAREN,
        E_PARSE_INVALID_TOKEN,
        "rachna",
        "'(' expected",
        "example: rachna add() { }"))
        return NULL;

    skip_separators();

    /* params */
    ASTNode **params = NULL;
    int param_count = 0;

    if (current.type != TOKEN_RIGHT_PAREN) {
        while (1) {
            if (current.type != TOKEN_IDENTIFIER) {
                shriji_error_at(current, E_PARSE_INVALID_TOKEN,
                    "rachna",
                    "parameter name expected",
                    "example: rachna add(a,b) { }");
                return NULL;
            }

            char p[128];
            strncpy(p, current.start, current.length);
            p[current.length] = '\0';

            ASTNode **tmp = realloc(params, sizeof(ASTNode*) * (param_count + 1));
            if (!tmp) return NULL;
            params = tmp;

            params[param_count++] = new_identifier_node(p);

            advance();
            skip_separators();
        }
    }
}

```

[illegible]

```

        "mavi",
        "Declaration samajh li, par variable ka naam clear nahi hua.",
        "mavi x ya mavi x = 10"
    );
}

return NULL;
}

char name[128];
strncpy(name, current.start, current.length);
name[current.length] = 0;
advance();

skip_separators();

if (current.type == TOKEN_EQUAL) {
    advance();
    skip_separators();

    if (current.type == TOKEN_NEWLINE ||
        current.type == TOKEN_EOF ||
        current.type == TOKEN_RIGHT_BRACE) {

        if (!error_reported) {
            shriji_error_at(
                current,
                E_ASSIGN_02,
                "assignment",
                "'=' ke baad value missing hai",
                "example: mavi x = 10"
            );
        }

        return NULL;
    }

    /* FIX HERE */
    ASTNode *val = parse_expression(); // â\234\205 CHANGE

    if (!val) return NULL;

    skip_separators();
    return new_assignment_node(name, val);
}

return new_assignment_node(name, NULL);
}

static ASTNode *parse_logical_and(void)
{
    ASTNode *node = parse_comparison();

    while (current.type == TOKEN_AND)
    {
        char op = '&';
        advance();

        ASTNode *right = parse_comparison();

        node = new_binary_node(op, node, right);
    }

    return node;
}

```

[illegible]

[illegible]

```

    advance();

/* ४\237\232" STRICT DOUBLE OPERATOR BLOCK */

if (current.type == TOKEN_PLUS ||
    current.type == TOKEN_MINUS ||
    current.type == TOKEN_STAR ||
    current.type == TOKEN_SLASH ||
    current.type == TOKEN_AND ||
    current.type == TOKEN_OR)
{
    if (!error_reported)
    {
        char example[64];
        shriji_build_example(current_source, example, sizeof(example));

        shriji_error_at(
            current,
            E_PARSE_DOUBLE_OPERATOR,
            "expr",
            "Do operators ek saath aa gaye hain.",
            example
        );

        error_reported = 1;
    }

    /* HARD STOP  \200\224 no recovery */
    return NULL;
}

/* operator end detection */

if (current.type == TOKEN_NEWLINE ||
    current.type == TOKEN_EOF ||
    current.type == TOKEN_RIGHT_BRACE)
{
    if (!error_reported) {
        char example[64];
        shriji_build_example(current_source, example, sizeof(example));

        shriji_error_at(
            current,
            E_PARSE_OPERATOR_END,
            "expr",
            "Operator ke baad value missing hai",
            example
        );
    }

    return NULL;
}

    ASTNode *right = parse_term();
    if (!right) return NULL;

    n = new_binary_node(op, n, right);
}

return n;
}

static ASTNode *parse_term(void)
{

```

```

        if (error_reported)
            return NULL;

ASTNode *n = parse_unary();
if (!n) return NULL;

while (current.type == TOKEN_STAR ||
        current.type == TOKEN_SLASH ||
        current.type == TOKEN_MOD)
{
    char op =
        (current.type == TOKEN_STAR) ? '*' :
        (current.type == TOKEN_SLASH) ? '/' : '%';

    advance();

/* double operator detection */

if (current.type == TOKEN_STAR ||
    current.type == TOKEN_SLASH ||
    current.type == TOKEN_MOD)
{
if (!error_reported) {

    char example[64];
    shriji_build_example(current_source, example, sizeof(example));

    shriji_error_at(
        current,
        E_PARSE_DOUBLE_OPERATOR,
        "expr",
        "Do operators ek saath aa gaye hain.",
        example
    );

/* INTELLIGENCE ADD */
    snprintf(LAST_ERROR.expected, sizeof(LAST_ERROR.expected), "%s", "value");
    snprintf(LAST_ERROR.received, sizeof(LAST_ERROR.received), "%s", current.start);
}

    return NULL;
}

/* operator end detection for * / % */

    if (current.type == TOKEN_NEWLINE ||
        current.type == TOKEN_EOF ||
        current.type == TOKEN_RIGHT_BRACE)
    {
        if (!error_reported) {

            char example[64];
            shriji_build_example(current_source, example, sizeof(example));

            shriji_error_at(
                current,
                E_PARSE_OPERATOR_END,
                "expr",
                "Expression operator par end ho gaya",
                example
            );
        }

        skip_separators();
        return NULL;
    }
}

```

[illegible]


```

        current,
        E_PARSE_OPERATOR_CHAIN,
        "expr",
        "operator chain galat hai",
        example
    );

    /* INTELLIGENCE ADD */
    snprintf(LAST_ERROR.expected, sizeof(LAST_ERROR.expected), "%s", "value");
    snprintf(LAST_ERROR.received, sizeof(LAST_ERROR.received), "%s", current.start);
}

return NULL;

}

ASTNode *expr = parse_primary();
if (!expr) return NULL;

return new_unary_node(op, expr);
}

/* nahi (logical not) */
if (current.type == TOKEN_NAHI) {
    advance();

    ASTNode *expr = parse_unary();
    if (!expr) return NULL;

    return new_not_node(expr);
}

/* default primary */
ASTNode *node = parse_primary();
if (!node) return NULL;

/* detect missing operator: 6(5+9) */
if (current.type == TOKEN_LEFT_PAREN)
{
    if (!error_reported) {

        char example[64];
        shriji_build_example(current_source, example, sizeof(example));

        shriji_error_at(
            current,
            E_PARSE_MISSING_OPERATOR,
            "expr",
            "Do values ke beech operator missing hai",
            example
        );
    }

    return NULL;
}

return node;
}

```

[illegible]

[illegible]

```

    if (current.type == TOKEN_TRUE) {
        advance();
        return new_bool_node(1);
    }

    if (current.type == TOKEN_FALSE) {
        advance();
        return new_bool_node(0);
    }

    if (current.type == TOKEN_NUMBER) {

        char temp[64];
        int len = current.length;

        if (len >= 63)
            len = 63;

        strncpy(temp, current.start, len);
        temp[len] = '\0';

        double v = atof(temp);

        advance();

        return new_number_node(v);
    }

    if (current.type == TOKEN_STRING) {
        char s[256];
        int len = current.length;

        const char *p = current.start;
        if (len >= 2 && p[0] == '"' && p[len - 1] == '"') {
            p++;
            len -= 2;
        }

        unescape_into(s, sizeof(s), p, len);

        advance();
        return new_string_node(s);
    }

    /* GROUPING: (expr) */
    if (current.type == TOKEN_LEFT_PAREN)
    {
        advance(); // consume '('

        ASTNode *expr = parse_expression();
        if (!expr) return NULL;

        if (current.type != TOKEN_RIGHT_PAREN)
        {
            shriji_error_at(
                current,
                E_PARSE_UNMATCHED_PAREN,
                ")",
                "missing )",
                "(expr)"
            );
            return NULL;
        }

        advance(); // consume ')'

        return expr;
    }

```

```

/* list literal: [ ... ] */
if (current.type == TOKEN_LEFT_BRACKET) {
    ASTNode *lst = parse_list_literal();
    if (!lst) return NULL;
    return parse_postfix(lst);
}

/* dict literal: { ... } */
if (current.type == TOKEN_LEFT_BRACE) {
    ASTNode *d = parse_dict_literal();
    if (!d) return NULL;
    ASTNode *node = parse_postfix(d);
    if (!node) return NULL;
    return node;
}

if (current.type == TOKEN_IDENTIFIER) {
    char name[128];
    strncpy(name, current.start, current.length);
    name[current.length] = 0;
    advance();
}

/* function call: name(...) */
if (current.type == TOKEN_LEFT_PAREN) {
    advance(); /* consume '(' */
    skip_separators();

    ASTNode **args = NULL;
    int arg_count = 0;

    if (current.type != TOKEN_RIGHT_PAREN) {
        while (1) {
            ASTNode *arg = parse_expression();
            if (!arg) {
                free(args);
                return NULL;
            }

            ASTNode **tmp = realloc(args, sizeof(ASTNode*) * (arg_count + 1));
            if (!tmp) {
                free(args);
                return NULL;
            }
            args = tmp;
            args[arg_count++] = arg;

            skip_separators();

            if (current.type == TOKEN_RIGHT_PAREN)
                break;

            if (current.type != TOKEN_COMMA) {
                shriji_error_at(
                    current,
                    E_FUNCTION_PARAM_INVALID,
                    "call",
                    "expected ',' between arguments",
                    "example: add(5, 6)"
                );
                free(args);
                return NULL;
            }
        }
    }
}

```

[illegible]

```

while (1) {

    /* key must be string for v1 */
    if (current.type != TOKEN_STRING) {
        shriji_error_at(
            current,
            E_DICT_KEY_INVALID,
            "dict",
            "dict key must be string",
            "example: {\\"a\\": 1, \\"b\\": 2}"
        );
        free(keys);
        free(vals);
        return NULL;
    }

    /* parse key as string node */
    ASTNode *k = parse_primary();
    if (!k) {
        free(keys);
        free(vals);
        return NULL;
    }

    skip_separators();

    if (!expect(TOKEN_COLON, E_DICT_SYNTAX_ERROR, ":", "missing ':' ",
        "example: {\\"a\\": 1}")) {
        free(keys);
        free(vals);
        return NULL;
    }

    skip_separators();

    /* parse value expression */
    ASTNode *v = parse_expression();
    if (!v) {
        free(keys);
        free(vals);
        return NULL;
    }

    /* push pair */
    ASTNode **tmpk = realloc(keys, sizeof(ASTNode*) * (count + 1));
    if (!tmpk) {
        free(keys);
        free(vals);
        return NULL;
    }
    keys = tmpk;

    ASTNode **tmpv = realloc(vals, sizeof(ASTNode*) * (count + 1));
    if (!tmpv) {
        free(vals);
        return NULL;
    }
    vals = tmpv;

    keys[count] = k;
    vals[count] = v;
    count++;

    skip_separators();

    /* end? */
    if (current.type == TOKEN_RIGHT_BRACE)
        break;
}

```


[illegible]

```

EngineResult shriji_engine_execute(const char *input, Env *env)
{
    EngineResult res;
    engine_result_init(&res);

    /* 0\237\224\222 RESET GLOBAL ERROR STATE */
    error_reported = 0;

    if (!input)
    {
        res.status = ENGINE_PARSE_ERROR;
        return res;
    }

    /* ===== PARSE ===== */

    ASTNode *ast = parse_once(input);

    if (!ast || error_reported)
    {
        res.status = ENGINE_PARSE_ERROR;
        return res;
    }

    /* ===== RUNTIME ===== */

    ShrijiRuntime runtime;
    runtime_init(&runtime);

    Value result = eval(ast, env, &runtime);

    if (runtime.error_flag || error_reported)
    {
        res.status = ENGINE_RUNTIME_ERROR;

        value_free(&result);
        ast_free(ast);

        return res;
    }

    /* ===== SUCCESS ===== */

    res.result = value_copy(result);
    value_free(&result);

    ast_free(ast);

    res.status = ENGINE_OK;

    return res;
}

```

```

=====
FILE: ./src/nirman.c
=====
#include "../include/nirman.h"
#include <string.h>

/* =====
GLOBAL CONTEXT
===== */
NirmanContext NIRMAN_CTX = {0};

/* =====
LIFECYCLE
===== */

```

```

void nirman_start(void)
{
    NIRMAN_CTX.active = 1;

    /* reset flow */
    NIRMAN_CTX.flow_stage = 0;
    NIRMAN_CTX.goal[0] = '\0';
}

void nirman_stop(void)
{
    NIRMAN_CTX.active = 0;

    /* reset flow */
    NIRMAN_CTX.flow_stage = 0;
    NIRMAN_CTX.goal[0] = '\0';
}

int nirman_is_active(void)
{
    return NIRMAN_CTX.active;
}

/* =====
INTERNAL HELPERS
===== */

static void to_lower(char *s)
{
    for (int i = 0; s[i]; i++)
    {
        if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] += 32;
    }
}

/* =====
INTENT ENGINE
===== */

int nirman_detect_intent(const char *input)
{
    if (!input)
        return 0;

    char temp[256];
    strncpy(temp, input, sizeof(temp) - 1);
    temp[255] = '\0';

    to_lower(temp);

    /* 1 = TALK */
    if (strstr(temp, "kaise") ||
        strstr(temp, "hello") ||
        strstr(temp, "hi") ||
        strstr(temp, "hy") ||
        strstr(temp, "radhe"))
    {
        return 1;
    }

    /* 2 = BUILD */
    if (strstr(temp, "bana") ||
        strstr(temp, "create") ||
        strstr(temp, "app") ||
        strstr(temp, "software"))
    {
        return 2;
    }
}

```

```
/* 3 = ERROR / HELP */
if (strstr(temp, "error") ||
    strstr(temp, "problem") ||
    strstr(temp, "issue"))
{
    return 3;
}

return 0;
}
```

```
=====
FILE: ./src/fix_rules.c
```

```
#include "../include/fix_rules.h"
#include "../include/fix_interactive.h"
```

```
#include <string.h>
#include <stdio.h>
#include <ctype.h>
```

[illegible][illegible]

```

/* FIX: double operator (6++6 â\206\222 6+6) */
static int fix_double_operator(
    char *buffer,
    size_t buffer_size,
    const ShrijiErrorInfo *err)
{
    (void)buffer_size;
    (void)err;

    int changed = 0;

    for (int i = 0; buffer[i]; i++)
    {
        if ((buffer[i] == '+' || buffer[i] == '-' ||
            buffer[i] == '*' || buffer[i] == '/')
            && buffer[i] == buffer[i+1])
        {
            memmove(&buffer[i], &buffer[i+1], strlen(&buffer[i+1]) + 1);
            changed = 1;
            i--;
        }
    }

    return changed;
}

```

```

/* FIX: missing operator (6(7+7) â\206\222 6*(7+7)) */
static int fix_missing_operator(
    char *buffer,
    size_t buffer_size,
    const ShrijiErrorInfo *err)
{
    (void)err;
}

```

[illegible]

[illegible]

```

    LIFECYCLE
    ----- */
void smriti_init(void)
{
    has_cont = 0;
    cont_val = 0.0;
    cont_op = 0;

    turn_counter = 0;
    last_input_buf[0] = '\0';
    last_intent_val = 0;
}

/* -----
CONTINUATION (AS-IS)
----- */
int smriti_has_continuation(void)
{
    return has_cont;
}

void smriti_set_continuation(double value, char op)
{
    has_cont = 1;
    cont_val = value;
    cont_op = op;
}

void smriti_clear_continuation(void)
{
    has_cont = 0;
    cont_val = 0.0;
    cont_op = 0;
}

double smriti_get_continuation_value(void)
{
    return cont_val;
}

char smriti_get_continuation_op(void)
{
    return cont_op;
}

/* -----
STEP-11 CONTEXT (PASSIVE)
----- */
void smriti_next_turn(void)
{
    turn_counter++;
}

unsigned long smriti_get_turn(void)
{
    return turn_counter;
}

void smriti_set_last_input(const char *text)
{
    if (!text)
    {
        last_input_buf[0] = '\0';
        return;
    }
    /* safe snapshot */
    strncpy(last_input_buf, text, sizeof(last_input_buf) - 1);
    last_input_buf[sizeof(last_input_buf) - 1] = '\0';
}

```

```

const char* smriti_get_last_input(void)
{
    return last_input_buf;
}

```

```

void smriti_set_last_intent(int intent)
{
    last_intent_val = intent;
}

```

```

int smriti_get_last_intent(void)
{
    return last_intent_val;
}

```

```

=====
FILE: ./src/translator.c
=====

```

```

#include <string.h>
#include <stdlib.h>
#include <ctype.h>
#include "../include/translator.h"

```

```

// -----
// INTERNAL STRUCT FOR KEYWORD MAPPING
// -----
typedef struct {
    const char *world;
    const char *shriji;
} TranslateEntry;

```

```

// -----
// WORLD â\206\222 SHRIJILANG KEYWORD MAP (SMART ROUTING V3)
// -----
static TranslateEntry keyword_map[] = {

```

```

    // PRINT / OUTPUT
    {"print", "aaradhya"},
    {"echo", "aaradhya"},
    {"say", "sakhi"},
    {"speak", "sakhi"},
    {"tell", "sakhi"},

```

```

    // MIRA â\200\224 Mathematical Solver
    {"solve", "mira"},
    {"calculate", "mira"},
    {"compute", "mira"},

```

```

    // SHIRI â\200\224 Soft Guidance
    {"guide", "shiri"},
    {"help", "shiri"},
    {"direction", "shiri"},

```

```

    // NIYU â\200\224 Logic / Why AI
    {"why", "niyu"},
    {"explain", "niyu"},
    {"reason", "niyu"},

```

```

    // KAVYA â\200\224 Creative Engine
    {"create", "kavya"},
    {"write", "kavya"},
    {"story", "kavya"},

```

```

    // DEVOTIONAL â\206\222 DIRECT KAVYA
    {"radhe", "kavya"},
    {"hare", "kavya"},
    {"krishna", "kavya"},

```



```

    {"shyam", "kavya"},

    // VARIABLE SYSTEM
    {"var", "mavi"},
    {"let", "mavi"},
    {"int", "mavi"},
    {"float", "mavi"}
};

// -----
// GET FIRST WORD
// -----
char* get_first_word(const char *source) {
    while (*source == ' ') source++;

    const char *start = source;
    while (*source && !isspace(*source))
        source++;

    int len = source - start;
    char *word = malloc(len + 1);
    strncpy(word, start, len);
    word[len] = '\0';

    return word;
}

// -----
// MAIN KEYWORD TRANSLATION
// -----
const char* translate_keyword(const char *word) {

    for (size_t i = 0; i < sizeof(keyword_map)/sizeof(keyword_map[0]); i++) {

        if (strcmp(word, keyword_map[i].world) == 0)
            return keyword_map[i].shriji;
    }

    return NULL;
}

// -----
// SMART AUTOCORRECT FOR NATURAL INPUT
// -----
const char* mira_autocorrect(const char *word) {

    // Logical â\206\222 Mira
    if (strcmp(word, "prit") == 0) return "mira";

    // Devotional â\206\222 Kavya
    if (strcmp(word, "radhe") == 0) return "kavya";
    if (strcmp(word, "hare") == 0) return "kavya";
    if (strcmp(word, "krishna") == 0) return "kavya";

    // Emotional pain â\206\222 Sakhi
    if (strcmp(word, "mera") == 0) return "sakhi";
    if (strcmp(word, "mann") == 0) return "sakhi";
    if (strcmp(word, "toot") == 0) return "sakhi";
    if (strcmp(word, "dukhi") == 0) return "sakhi";
    if (strcmp(word, "hurt") == 0) return "sakhi";
    if (strcmp(word, "sad") == 0) return "sakhi";

    // Creative autocorrect
    if (strcmp(word, "creat") == 0) return "kavya";

    // Shiri autocorrect
    if (strcmp(word, "gide") == 0) return "shiri";

    return NULL;
}

```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```

if (peek() == '\0')
    return make_token(TOKEN_EOF, start, 0);

char c = advance();

if (isdigit(c))
    return number();

if (isalpha(c) || c == '_')
    return identifier();

switch (c) {

    case '+':
        if (match('+')) return make_token(TOKEN_PLUSPLUS, start, 2);
        return make_token(TOKEN_PLUS, start, 1);

    case '-':
        if (match('-')) return make_token(TOKEN_MINUSMINUS, start, 2);
        return make_token(TOKEN_MINUS, start, 1);

    case '*': return make_token(TOKEN_STAR, start, 1);
    case '/': return make_token(TOKEN_SLASH, start, 1);

    case '&':
        if (match('&')) return make_token(TOKEN_AND, start, 2);
        return make_token(TOKEN_ERROR, start, 1);

    case '|':
        if (match('|')) return make_token(TOKEN_OR, start, 2);
        return make_token(TOKEN_ERROR, start, 1);

    case '%': return make_token(TOKEN_MOD, start, 1);

    case '(': return make_token(TOKEN_LEFT_PAREN, start, 1);
    case ')': return make_token(TOKEN_RIGHT_PAREN, start, 1);
    case '{': return make_token(TOKEN_LEFT_BRACE, start, 1);
    case '}': return make_token(TOKEN_RIGHT_BRACE, start, 1);
    case '[': return make_token(TOKEN_LEFT_BRACKET, start, 1);
    case ']': return make_token(TOKEN_RIGHT_BRACKET, start, 1);

    case ',': return make_token(TOKEN_COMMA, start, 1);
    case ':': return make_token(TOKEN_COLON, start, 1);

    case ';': return make_token(TOKEN_SEMICOLON, start, 1);

    case '=':
        return make_token(match('=') ? TOKEN_EQEQ : TOKEN_EQUAL, start, 1);

    case '!':
        return make_token(match('=') ? TOKEN_NEQ : TOKEN_NOT, start, 1);

    case '>':
        return make_token(match('=') ? TOKEN_GTE : TOKEN_GT, start, 1);

    case '<':
        return make_token(match('=') ? TOKEN_LTE : TOKEN_LT, start, 1);

    case '\n':
        return make_token(TOKEN_NEWLINE, start, 1);

    case '"':
        return string();

}

/* Unknown / Unicode characters skip */
return make_token(TOKEN_ERROR, start, 1);
}

```



```

        break;

/* ===== ERROR ===== */

case KRST_RESPONSE_ERROR:

    printf("ERROR:\n");

    if (response->message[0] != '\0')
    {
        printf("%s\n\n", response->message);
    }

    if (response->details[0] != '\0')
    {
        printf("DETAILS:\n");
        printf("%s\n\n", response->details);
    }

    if (response->input[0] != '\0')
    {
        printf("INPUT:\n");
        printf("%s\n", response->input);

        if (response->pointer_col > 0)
        {
            for (int i = 1; i < response->pointer_col; i++)
                printf(" ");

            printf("^\n");
        }

        printf("\n");
    }

    if (response->hint[0] != '\0')
    {
        printf("SUGGESTION:\n");
        printf("%s\n", response->hint);
    }

    break;

/* ===== WARNING ===== */

case KRST_RESPONSE_WARNING:

    printf("WARNING:\n");

    if (response->message[0] != '\0')
    {
        printf("%s\n\n", response->message);
    }

    if (response->details[0] != '\0')
    {
        printf("DETAILS:\n");
        printf("%s\n\n", response->details);
    }

    break;

/* ===== CRITICAL ===== */

case KRST_RESPONSE_CRITICAL:

    printf("CRITICAL:\n");

```

```

    if (response->message[0] != '\0')
    {
        printf("%s\n\n", response->message);
    }

    if (response->details[0] != '\0')
    {
        printf("DETAILS:\n");
        printf("%s\n", response->details);
    }

    break;

```

```
case KRST_RESPONSE_AUTO_FIX:
```

```

if (
    response->original_input[0] == '\0' &&
    response->corrected_input[0] == '\0'
)
{
    break;
}

printf("AUTO FIX:\n");

if (response->original_input[0] != '\0')
{
    printf("INPUT:\n");
    printf("%s\n\n", response->original_input);
}

if (response->corrected_input[0] != '\0')
{
    printf("CORRECTED:\n");
    printf("%s\n\n", response->corrected_input);
}

if (response->result_text[0] != '\0')
{
    printf("OUTPUT:\n");
    printf("%s\n", response->result_text);
}

break;

default:
    break;
}

}

```

```

=====
FILE: ./krst/krst_router.c
=====

```

```

#include <stdio.h>
#include <string.h>

```

```

#include "../include/krst_router.h"
#include "../krst/krst_core.h"
#include "../include/pragya_avastha.h"

```

```

#include "../include/error.h"
#include "../include/error_intelligence.h"

```

```

/* =====
INPUT NORMALIZATION (SAFE)
===== */

```

```

static void normalize_input(const char *input, char *output)
{
    if (!input || !output) return;

    int i = 0;
    for (; input[i]; i++) {
        if (input[i] >= 'A' && input[i] <= 'Z')
            output[i] = input[i] + 32;
        else
            output[i] = input[i];
    }
    output[i] = '\0';
}

/* =====
COMMAND DETECTION
===== */
static char detect_command(const char *input)
{
    if (!input) return 'G';

    if (input[0] == 'g' && input[1] == ' ')
        return 'G';

    if (input[0] == 'e' && input[1] == ' ')
        return 'E';

    return 'G';
}

/* =====
CONTEXT REWRITE (C2 COMPLETE)
===== */
static int rewrite_context_input(KRSTRequest *req)
{
    if (!req || !req->input_text) return 0;

    int num = 0;
    static char buffer[128];

    const char *input = req->input_text;

    /* ===== ADD ===== */
    if ((strstr(input, "add") || strstr(input, "jodo")) &&
        sscanf(input, "usme %d", &num) == 1) {

        snprintf(buffer, sizeof(buffer), "bolo last + %d", num);
        req->input_text = buffer;
        return 1;
    }

    /* ===== MINUS ===== */
    if ((strstr(input, "minus") || strstr(input, "ghatao") || strstr(input, "subtract")) &&
        (sscanf(input, "usme %d", &num) == 1 ||
         sscanf(input, "usme se %d", &num) == 1)) {

        snprintf(buffer, sizeof(buffer), "bolo last - %d", num);
        req->input_text = buffer;
        return 1;
    }

    /* ===== MULTIPLY ===== */
    if ((strstr(input, "multiply") || strstr(input, "guna") || strstr(input, "into")) &&
        sscanf(input, "usme %d", &num) == 1) {

        snprintf(buffer, sizeof(buffer), "bolo last * %d", num);
        req->input_text = buffer;
        return 1;
    }
}

```

```

/* ===== DIVIDE ===== */
if ((strstr(input, "divide") || strstr(input, "bhaag") || strstr(input, "divided")) &&
    (sscanf(input, "usme %d", &num) == 1 ||
     sscanf(input, "usme se %d", &num) == 1)) {

    snprintf(buffer, sizeof(buffer), "bolo last / %d", num);
    req->input_text = buffer;
    return 1;
}

return 0;
}

/* =====
MAIN ROUTER
===== */
void krst_route_request(KRSTRequest *req)
{
    if (!req || !req->input_text)
        return;

    /* ===== SAFE NORMALIZATION ===== */
    static char normalized[256];

    normalize_input(req->input_text, normalized);
    req->input_text = normalized;

    /* ===== COMMAND DETECTION ===== */
    char cmd = detect_command(req->input_text);

    /* ===== COMMAND ROUTING ===== */

    if (cmd == 'E') {
        req->input_text += 2;

        static char explain_buf[256];
        snprintf(explain_buf, sizeof(explain_buf), "bolo %.240s", req->input_text);
        req->input_text = explain_buf;

        printf("[EXPLAIN MODE]\n");
    }

    else if (cmd == 'G') {
        if (req->input_text[0] == 'g' && req->input_text[1] == ' ')
            req->input_text += 2;
    }

    /* ===== CONTEXT REWRITE ===== */
    rewrite_context_input(req);

    /* ===== CREATE AVASTHA ===== */
    PragyaAvastha avastha;
    memset(&avastha, 0, sizeof(PragyaAvastha));

    avastha.raw_text = req->input_text;

    /* NEW: pass input mode */
    avastha.is_program_mode = req->is_program_mode;

    /* ===== KRST CORE ===== */
    krst_process(&avastha);

    /* ===== ERROR INTELLIGENCE FALLBACK (FIXED) ===== */
    if (!avastha.has_correction && error_reported)
    {
        extern int GYAAN_ALREADY_PRINTED;

        /* 0\237\224\222 duplicate guard */

```

```

    if (!GYAAN_ALREADY_PRINTED)
    {
        shriji_error_intelligence(&avastha, &LAST_ERROR, NULL);
        GYAAN_ALREADY_PRINTED = 1;
    }
}

/* ===== RETURN CORRECTION ===== */
if (avastha.has_correction)
{
    req->has_correction = 1;

    strncpy(req->corrected_text,
            avastha.corrected_text,
            sizeof(req->corrected_text) - 1);

    req->corrected_text[
        sizeof(req->corrected_text) - 1] = '\0';

    /* ===== PROPAGATE STOP EXECUTION ===== */
    req->stop_execution = avastha.stop_execution;
}
}

```

```

=====
FILE: ./krst/krst_core.c
=====

```

```

/* RADHE_RADHE_SHREEJI_RADHE_RANI */

```

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

```

```

#include "../include/interpreter.h"
#include "../include/error.h"
#include "../include/fix_engine.h"
#include "../include/fix_rules.h"

#include "../include/nirman.h"
#include "../include/parser.h"
#include "../include/value.h"
#include "../include/env.h"
#include "../include/runtime.h"
#include "../include/pragya_avastha.h"
#include "../include/decision_engine.h"

```

```

#include "krst_core.h"
#include "krst_types.h"

```

```

#include "../pragya/router/l3_router.h"
#include "../include/error_intelligence.h"

```

```

#include "../include/global.h"
#include "../include/user_config.h"

```

```

#include "../gyaan/core/gyaan_engine.h"
#include "../include/krst_print.h"

```

```

extern Env *GLOBAL_ENV;

```

```

/* ===== SESSION ===== */

```

```

static int session_confidence = 100;
static int session_risk = 4;
static int success_streak = 0;

```

```

#define MAX_ERROR_TYPES 50

```

[illegible]


```

        "%s",
        avastha->raw_text
    );

    response.pointer_col = err->col;
}

/* HINT */
if (strlen(err->hint) > 0)
{
    snprintf(
        response.hint,
        sizeof(response.hint),
        "%s",
        err->hint
    );
}

response.type = response_type;

/* 1. Base Error (always) */

/* CORE INTELLIGENCE SUGGESTION */
if (strcmp(err->expected, "value") == 0)
{
    if (strcmp(err->received, "+") == 0 ||
        strcmp(err->received, "-") == 0 ||
        strcmp(err->received, "*") == 0 ||
        strcmp(err->received, "/" ) == 0)
    {
        snprintf(
            response.details,
            sizeof(response.details),
            "extra operator remove karo"
        );
    }
}

/* 2. GYAAN fetch */
const char *gyaan = gyaan_get(err);

/* EXPLAIN LEVEL */
if (avastha->teach_level >= KRST_TEACH_EXPLAIN)
{
    if (gyaan &&
        strcmp(gyaan, err->message) != 0)
    {
        snprintf(
            response.details,
            sizeof(response.details),
            "%s",
            gyaan
        );
    }
}

/* DEEP LEVEL */
if (avastha->teach_level >= KRST_TEACH_DEEP)
{
    shriji_error_intelligence(avastha, err, NULL);
}

/* TRAIN LEVEL (future expandable) */
if (avastha->teach_level >= KRST_TEACH_TRAIN)
{

```



```

        snprintf(
            response.details,
            sizeof(response.details),
            "Practice karo: similar expression likho aur test karo"
        );
    }

    krst_print_response(&response);
    /* 4. Stop execution */
    avastha->stop_execution = 1;
}

/* ===== THRESHOLD ===== */
static void apply_thresholds(KRSTDecision *d)
{
    int c = d->confidence_score;

    if (c <= 20)        d->teaching_level = KRST_TEACH_TRAIN;
    else if (c <= 40)    d->teaching_level = KRST_TEACH_DEEP;
    else if (c <= 60)    d->teaching_level = KRST_TEACH_EXPLAIN;
    else if (c <= 80)    d->teaching_level = KRST_TEACH_HINT;
    else                 d->teaching_level = KRST_TEACH_SILENT;

    if (session_risk >= 76)
    {
        d->tone = KRST_TONE_ALERT;
        d->escalate = 1;
    }
    else if (session_risk >= 30)
    {
        d->tone = KRST_TONE_STRICT;
        d->escalate = 0;
    }
    else
    {
        d->tone = KRST_TONE_NEUTRAL;
        d->escalate = 0;
    }
}

/* ===== MAIN ===== */

EngineResult krst_process(PragyaAvastha *avastha)
{
    EngineResult res;
    engine_result_init(&res);

    if (!avastha || !avastha->raw_text)
    {
        res.status = ENGINE_PARSE_ERROR;
        return res;
    }

    const char *input = avastha->raw_text;

    /* NIRMAN START */
    if (strcmp(input, "nirman") == 0) {
        nirman_start();
        printf("Nirman mode activated\n");
        printf("Welcome to Shriji World\n");
        res.status = ENGINE_OK;
        return res;
    }

    /* NIRMAN STOP */
    if (strcmp(input, "exit nirman") == 0) {
        nirman_stop();
    }
}

```

```

        printf("Nirman mode deactivated\n");
        res.status = ENGINE_OK;
        return res;
    }

    /* NIRMAN MODE */
    if (nirman_is_active()) {
        int intent = nirman_detect_intent(input);
        printf("SHRIJI (intent=%d)\n", intent);
        res.status = ENGINE_OK;
        return res;
    }

    if (DEV_MODE)
        printf("[KRST] Input: %s\n", input);

    error_reported = 0;
    avastha->stop_execution = 0;
    avastha->allow_auto_fix = 0;

/* ॐ\237\224» RUN ENGINE */
EngineResult engine_res =
    shriji_engine_execute(input, GLOBAL_ENV);
/* ॐ\237\224» ERROR HANDLE */
const ShrijiErrorInfo *err_ptr = NULL;

if (engine_res.status != ENGINE_OK)
{
    err_ptr = shriji_last_error();
}

if (err_ptr)
{
    ShrijiErrorInfo err_safe = *err_ptr;
    error_reported = 0;

    int repeat = update_error_memory(err_safe.code);

    /* FIX RULE LOOKUP */
    FixRule *rule =
        shriji_get_rule_for_error(err_safe.code);

    /* RESPONSE TYPE DECISION */
    KRSTResponseType response_type =
        krst_decide_response_type(repeat, rule);

    /* ॐ\237\224» ADD THIS BLOCK BELOW */
    session_confidence -= 10;

    if (session_confidence < 0)
        session_confidence = 0;

    switch (response_type)
    {
        case KRST_RESPONSE_ERROR:

            avastha->teach_level = KRST_TEACH_EXPLAIN;
            avastha->risk = 20;
            avastha->allow_auto_fix = 0;

            break;

        case KRST_RESPONSE_WARNING:

            avastha->teach_level = KRST_TEACH_DEEP;
            avastha->risk = 50;
            avastha->allow_auto_fix = 0;

```

```

        break;

case KRST_RESPONSE_AUTO_FIX:

    avastha->teach_level = KRST_TEACH_DEEP;
    avastha->risk = 50;
    avastha->allow_auto_fix = 1;

    break;

case KRST_RESPONSE_CRITICAL:

    avastha->teach_level = KRST_TEACH_TRAIN;
    avastha->risk = 80;
    avastha->allow_auto_fix = 0;

    break;

default:
    break;
}

/* CENTRAL ERROR HANDLER */
handle_error(
    avastha,
    &err_safe,
    response_type
);

/* FIX ENGINE RE-EXECUTION */
if (avastha->has_correction && avastha->allow_auto_fix)
{
    if (DEV_MODE)
    {
        printf("[KRST] Applying auto-correction...\n");
    }

    EngineResult final_res = shriji_engine_execute(
        avastha->corrected_text,
        GLOBAL_ENV
    );

    /* APPLY FIXED RESULT */
    if (final_res.status == ENGINE_OK)
    {
        value_free(&engine_res.result);
        engine_res.result = value_copy(final_res.result);

        value_free(&final_res.result);
        engine_res.status = ENGINE_OK;
        avastha->stop_execution = 0; // allow normal flow
    }
    else
    {
        avastha->stop_execution = 1;
    }
}

}

/* Ø\237\237¢ SUCCESS TRACKING */
if (engine_res.status == ENGINE_OK)
{
    success_streak++;

    /* Ø\237\224» RESET ONLY AFTER STRONG PROOF */
    if (success_streak >= 5)
    {
        error_memory_size = 0;
        session_confidence = 100;
    }
}

```

```

        avastha->risk = 5;
        avastha->teach_level = KRST_TEACH_SILENT;
    }
}
else
{
    /* 0\237\224» DON'T HARD RESET */
    if (success_streak > 0)
        success_streak--;
}

if (engine_res.status == ENGINE_OK)
{
    /* AUTO FIX */
    if (avastha->allow_auto_fix)
    {
        KRSTResponse response;
        memset(&response, 0, sizeof(response));

        response.type = KRST_RESPONSE_AUTO_FIX;

        snprintf(
            response.original_input,
            sizeof(response.original_input),
            "%s",
            avastha->raw_text
        );

        snprintf(
            response.corrected_input,
            sizeof(response.corrected_input),
            "%s",
            avastha->corrected_text
        );

        if (engine_res.result.type == VAL_NUMBER)
        {
            snprintf(
                response.result_text,
                sizeof(response.result_text),
                "%g",
                engine_res.result.number
            );
        }
        else if (
            engine_res.result.type == VAL_STRING &&
            engine_res.result.string
        )
        {
            snprintf(
                response.result_text,
                sizeof(response.result_text),
                "%s",
                engine_res.result.string
            );
        }

        krst_print_response(&response);

        return engine_res;
    }

    /* NORMAL OUTPUT */
    KRSTResponse response;
    memset(&response, 0, sizeof(response));

    response.type = KRST_RESPONSE_NORMAL;

```

```

if (engine_res.result.type == VAL_NUMBER)
{
    snprintf(
        response.result_text,
        sizeof(response.result_text),
        "%g",
        engine_res.result.number
    );
}
else if (
    engine_res.result.type == VAL_STRING &&
    engine_res.result.string
)
{
    snprintf(
        response.result_text,
        sizeof(response.result_text),
        "%s",
        engine_res.result.string
    );
}
else if (engine_res.result.type == VAL_BOOL)
{
    snprintf(
        response.result_text,
        sizeof(response.result_text),
        "%s",
        engine_res.result.boolean ? "true" : "false"
    );
}
else
{
    snprintf(
        response.result_text,
        sizeof(response.result_text),
        "null"
    );
}

krst_print_response(&response);

}
return engine_res;
}

```

```

=====
FILE: ./krst/krst_types.h
=====
#ifndef KRST_TYPES_H
#define KRST_TYPES_H

#include "../include/env.h"

/* =====
   KRST ACTION ENUM
===== */

typedef enum {
    KRST_ACTION_ALLOW = 0,
    KRST_ACTION_BLOCK,
    KRST_ACTION_TEACH,
    KRST_ACTION_ESCALATE
} KRSTAction;

/* =====
   KRST TEACHING LEVEL ENUM

```

```

===== */

typedef enum {
    KRST_TEACH_SILENT = 0,
    KRST_TEACH_HINT,
    KRST_TEACH_EXPLAIN,
    KRST_TEACH_DEEP,
    KRST_TEACH_TRAIN
} KRSTTeachingLevel;

/* =====
   KRST TONE ENUM
===== */

typedef enum {
    KRST_TONE_NEUTRAL = 0,
    KRST_TONE_SUPPORTIVE,
    KRST_TONE_GUIDING,
    KRST_TONE_STRICT,
    KRST_TONE_ALERT
} KRSTTone;

/* =====
   KRST REQUEST STRUCT
===== */
typedef struct
{
    const char *input_text;

    int is_program_mode;
    /* correction support */

    char corrected_text[512];
    int has_correction;
    int stop_execution;
} KRSTRequest;

/* =====
   KRST DECISION STRUCT
===== */

typedef struct {
    KRSTAction action;

    int confidence_score; /* 0â\200\223100 */
    int risk_score;      /* 0â\200\223100 */

    int allow_execution;
    int reason_code;

    KRSTTeachingLevel teaching_level;
    KRSTTone tone;

    int escalate; /* 1 = escalation required */
} KRSTDecision;

/* =====
   KRST RESPONSE TYPE
===== */

typedef enum {
    KRST_RESPONSE_NORMAL = 0,
    KRST_RESPONSE_ERROR,
    KRST_RESPONSE_WARNING,
    KRST_RESPONSE_AUTO_FIX,
    KRST_RESPONSE_CRITICAL
} KRSTResponseType;

```

```
/* =====  
KRST RESPONSE STRUCT  
===== */
```

```
typedef struct {  
  
    KRSTResponseType type;  
  
    char message[256];  
    char details[512];  
    char cause[256];  
  
    char input[512];  
    int pointer_col;  
  
    char hint[256];  
  
    char result_text[256];  
  
    char original_input[512];  
    char corrected_input[512];  
  
    int repeat_count;  
  
} KRSTResponse;
```

```
#endif
```

```
=====
```

```
FILE: ./krst/krst_core.h
```

```
=====
```

```
#ifndef KRST_CORE_H  
#define KRST_CORE_H
```

```
#include "../include/engine.h"  
#include "../include/pragya_avastha.h"
```

```
/* Main processing function */  
EngineResult krst_process(PragyaAvastha *avastha);
```

```
#endif
```

```
=====
```

```
FILE: ./krst/niyu_engine.c
```

```
=====
```

```
#include <string.h>  
#include "niyu_engine.h"
```

```
int niyu_analyze(const char *input)  
{  
    if (!input)  
        return 0;  
  
    if (strlen(input) == 0)  
        return 0;  
  
    return 0;  
}
```

```
=====
```

```
FILE: ./Makefile
```

```
=====
```

```
# =====  
# SHRIJILANG â\200\224 INCREMENTAL BUILD SYSTEM  
# =====
```

CC = gcc

CFLAGS = -Wall -Wextra -std=c11 -O2 \
-Iinclude \
-Isrc \
-Ienv \
-Ipragya \
-Ikrst

OUT = shrijilang

=====
SOURCE FILES
=====

SRC = \
src/token.c \
src/ast.c \
src/parser.c \
gyaan/core/gyaan_engine.c \
gyaan/data/gyaan_rules.c \
src/interpreter.c \
src/runtime.c \
src/error.c \
src/state.c \
src/event.c \
src/command.c \
src/translator.c \
src/example_builder.c \
src/ai_router.c \
src/ai_intent.c \
src/atma_lekha.c \
src/smriti.c \
src/smriti_session.c \
src/smriti_personal.c \
src/main.c \
env/env.c \
src/lang/grammar/grammar_registry.c \
src/lang/grammar/grammar_core.c \
pragya/router/l3_router.c \
pragya/niyu/niyu_core.c \
pragya/sakhi/sakhi_core.c \
pragya/mira/mira_core.c \
krst/krst_core.c \
krst/krst_router.c \
krst/niyu_engine.c \
krst/krst_print.c \
src/fix_engine.c \
src/fix_rules.c \
src/fix_interactive.c \
src/typo_engine.c \
src/error_intelligence.c \
src/decision_engine.c \
src/user_config.c \
src/nirman.c \
src/normalize_engine.c \
src/nirman_flow_engine.c \
src/value_print.c

=====
OBJECT FILES
=====

OBJ = \$(SRC:.c=.o)

=====
FINAL BUILD
=====


```

$(OUT): $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $(OUT) -lreadline

# =====
# OBJECT BUILD RULE
# =====

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# =====
# CLEAN
# =====

clean:
    rm -f $(OUT)
    find . -name "*.o" -delete

# =====
# RUN
# =====

run: $(OUT)
    ./$(OUT)

# =====
# REBUILD
# =====

rebuild: clean $(OUT)

=====
FILE: ./pragya/sakhi/sakhi_core.c
=====
#include "sakhi_core.h"
#include <stdio.h>

L3Response sakhi_speak(const L3Request *request,
                      const NiyuResult *result)
{
    (void)result;

    L3Response res;

    // Input validation
    if (!request || !request->raw_input)
    {
        res.text = "Invalid input";
        res.success = 0;
        return res;
    }

    // Minimal clean log (optional, can remove later)
    printf("[PRAGYA] Processing input...\n");

    // Pass-through response
    res.text = request->raw_input;
    res.success = 1;

    return res;
}

=====
FILE: ./pragya/sakhi/sakhi_core.h
=====
#ifndef SAKHI_CORE_H
#define SAKHI_CORE_H

```

```

#include "sakhi_contract.h"

/*
 * SAKHI CORE
 *
 * Implementation file will:
 * - handle tone / language / clarity
 * - never inspect raw logic
 *
 * This header intentionally exposes no helpers.
 */

#endif /* SAKHI_CORE_H */

=====
FILE: ./pragya/sakhi/sakhi_contract.h
=====
#ifndef SAKHI_CONTRACT_H
#define SAKHI_CONTRACT_H

#include "../contracts/l3_request.h"
#include "../contracts/l3_response.h"
#include "../niyu/niyu_contract.h"

/*
 * SAKHI CONTRACT
 *
 * Sakhi is the voice of the system.
 * - Receives decided intent + thinking result (opaque)
 * - Produces final human-readable response
 *
 * Sakhi never:
 * - thinks or calculates
 * - decides intent
 * - teaches (that is Mira)
 */

/* Forward declaration (opaque thinking result) */
/*
 * sakhi_speak
 *
 * Final response generator.
 * - Called only by Router
 * - Uses intent already decided
 * - Formats language and tone
 */
L3Response sakhi_speak(const L3Request *request,
                      const NiyuResult *result);

#endif /* SAKHI_CONTRACT_H */

=====
FILE: ./pragya/contracts/l3_intent.h
=====
#ifndef L3_INTENT_H
#define L3_INTENT_H

/*
 * L3 Intent:
 * Router decides intent.
 * Niyu never decides intent.
 * Sakhi/Mira never guess intent.
 */

typedef enum {
    L3_INTENT_UNKNOWN = 0,

```

```

/* Explanation / reasoning */
L3_INTENT_EXPLAIN,
L3_INTENT_WHY,

/* Pure calculation / logic */
L3_INTENT_CALC,

/* Teaching / learning */
L3_INTENT_TEACH,

/* Emotional / tone-based */
L3_INTENT_EMOTION,

/* Explicit command */
L3_INTENT_COMMAND

} L3Intent;

#endif /* L3_INTENT_H */

=====
FILE: ./pragya/contracts/l3_rules.h
=====
#ifndef L3_RULES_H
#define L3_RULES_H

/*
 * L3 NON-NEGOTIABLE LAWS
 *
 * 1. Router decides intent.
 * 2. Niyu THINKS, never SPEAKS.
 * 3. Sakhi SPEAKS, never THINKS.
 * 4. Mira TEACHES only when intent == L3_INTENT_TEACH.
 *    No external permission, no memory switch.
 * 5. No direct Niyu <-> Sakhi calls.
 * 6. All communication flows through Router.
 * 7. Contracts are stable forever.
 * 8. Engine may change, behavior must not.
 */

#define L3_RULE_ROUTER_IS_AUTHORITY 1
#define L3_RULE_NIYU_SILENT 1
#define L3_RULE_SAKHI_NO_LOGIC 1
#define L3_RULE_MIRA_EXPLICIT_ONLY 1

#endif /* L3_RULES_H */

=====
FILE: ./pragya/contracts/l3_request.h
=====
#ifndef L3_REQUEST_H
#define L3_REQUEST_H

#include "l3_intent.h"

/*
 * L3Request:
 * Immutable input structure.
 * Router decides intent.
 * No teaching_mode flag here.
 * Behavior intent se hi control hoga.
 */

typedef struct {
    const char *raw_input; /* original user text */
    L3Intent intent; /* decided by router */

```

```

} L3Request;

#endif /* L3_REQUEST_H */

=====
FILE: ./pragya/contracts/l3_response.h
=====
#ifndef L3_RESPONSE_H
#define L3_RESPONSE_H

/*
 * L3Response:
 * Only Sakhi or Mira can create final response.
 * Niyu can never write directly here.
 */

typedef struct {
    const char *text;      /* final output */
    int         success;    /* logical success/failure */
} L3Response;

#endif /* L3_RESPONSE_H */

=====
FILE: ./pragya/mira/mira_core.c
=====
// JAI SHREE KRISHNA
#include "mira_contract.h"
#include "../niyu/niyu_contract.h"

#include "../../include/error.h"
#include "../../include/pragya_avastha.h"

#include <stdio.h>
#include <string.h>

/* =====
   EXPLAIN PRINT (CORE)
   ===== */
static void explain(const char *text)
{
    if (!text) return;
    printf("EXPLAIN: %s", text);
}

/* =====
   POINTER PRINTER
   ===== */
static void print_pointer(const char *text, int pos)
{
    if (!text || pos < 0)
        return;

    printf("EXPLAIN: %s\n", text);

    printf("EXPLAIN: ");
    for (int i = 0; i < pos; i++)
        printf(" ");

    printf("\n");
    printf("\n");
}

/* =====
   MIRA TEACHING ENGINE
   ===== */
void mira_teach_avastha(PragyaAvastha *a, NiyuResult *res)

```

```

{
    if (!a || !res)
        return;

    const char *text = a->raw_text;

    if (res->error_type == NIYU_ERR_VALID)
        return;

    switch (a->error_code)
    {
    case E_PARSE_DOUBLE_OPERATOR:
    case E_PARSE_OPERATOR_CHAIN:

        explain("Extra operator detect hua hai\n\n");

        print_pointer(text, res->error_pos);

        explain("Do operators ek saath allowed nahi hote.\n\n");
        explain("Example: 6 + 5\n");

        break;

    case E_PARSE_OPERATOR_START:

        explain("Expression operator se start nahi hota\n\n");

        print_pointer(text, res->error_pos);

        explain("Pehle ek value honi chahiye.\n\n");
        explain("Example: 6 + 5\n");

        break;

    case E_PARSE_OPERATOR_END:

        explain("Expression incomplete lag raha hai\n\n");

        print_pointer(text, res->error_pos);

        explain("Operator ke baad ek value expected hoti hai.\n\n");
        explain("Example: 6 + 5\n");

        break;

    case E_PARSE_MISSING_OPERATOR:

        explain("Operator missing hai\n\n");

        print_pointer(text, res->error_pos);

        explain("Do values ke beech operator hona chahiye.\n\n");
        explain("Example: 5 + 6\n");

        break;

    case E_ASSIGN_02:

        explain("Assignment incomplete hai\n\n");

        print_pointer(text, res->error_pos);

        explain("'=' ke baad value expected hai.\n\n");
        explain("Example: mavi x = 10\n");

        break;

    case E_PARSE_02:

```

```

        explain("Syntax error detect hua hai\n\n");

        print_pointer(text, res->error_pos);

        explain("Invalid token ya unsupported symbol mila hai.\n\n");
        explain("Example: 6 + 5\n");

        break;

    default:
        return;
    }
}

```

```

=====
FILE: ./pragya/mira/mira_contract.h
=====
#ifndef MIRA_CONTRACT_H
#define MIRA_CONTRACT_H

#include "../contracts/l3_request.h"
#include "../contracts/l3_response.h"
#include "../niyu/niyu_contract.h"
#include "../../include/pragya_avastha.h"

/*
 * MIRA CONTRACT
 *
 * Mira is the teaching layer.
 * - Activated only when teaching_mode == 1
 * - Provides structured explanation / lesson
 *
 * Mira never:
 * - handles normal conversation
 * - decides intent
 * - performs deep logic (that is Niyu)
 */

/*
 * Legacy interface (old L3 system)
 */
L3Response mira_teach(const L3Request *request);

/*
 * New ShrijiLang teaching interface
 * Used by Pragya Router
 */

void mira_teach_avastha(
    PragyaAvastha *avastha,
    NiyuResult *result
);

#endif /* MIRA_CONTRACT_H */

```

```

=====
FILE: ./pragya/mira/mira_core.h
=====
#ifndef MIRA_CORE_H
#define MIRA_CORE_H

#include "mira_contract.h"

/*
 * MIRA CORE
 *

```

```

* Implementation file will:
* - manage lesson flow
* - provide explanations
*
* No other module may call Mira directly.
*/

#endif /* MIRA_CORE_H */

=====
FILE: ./pragya/router/l3_router.c
=====
#include "l3_router.h"

#include "../niyu/niyu_contract.h"
#include "../sakhi/sakhi_contract.h"
#include "../mira/mira_contract.h"

#include "../../../include/pragya_avastha.h"

#include <stdio.h>

/*
=====
    PRAGYA ROUTER (L3)

    ROLE:
    - KRST se aayi hui avastha ko process karna
    - Niyu â\206\222 logic analysis
    - Mira â\206\222 teaching layer
    - Sakhi â\206\222 user response

    NOTE:
    - stop_execution flag ka respect MUST hai
=====
*/

void pragya_route(PragyaAvastha *avastha)
{
    /* ===== */
    /* BASIC SAFETY CHECK          */
    /* ===== */

    if (!avastha)
        return;

    /* ===== */
    /* ð\237\224¥ STOP EXECUTION GUARD */
    /* ===== */

    if (avastha->stop_execution)
        return;

    /* ===== */
    /* SILENT MODE                  */
    /* ===== */

    if (avastha->teach_level == 0)
        return;

    /* ===== */
    /* NIYU ANALYSIS                */
    /* ===== */

    NiyuResult *result = niyu_think_avastha(avastha);

    if (!result)
        return;

```

```

/* ===== */
/* MIRA TEACHING      */
/* ===== */

if (avastha->teach_level > 1)
{
    mira_teach_avastha(avastha, result);
}

/* ===== */
/* SAKHI RESPONSE      */
/* ===== */

if (avastha->error_code != 0)
{
    L3Request request;

    request.raw_input = avastha->raw_text;
    request.intent = 0;

    sakhi_speak(&request, result);
}

/* ===== */
/* CLEANUP              */
/* ===== */

niyu_free_result(result);
}

```

```

=====
FILE: ./pragya/router/l3_router.h
=====
#ifndef PRAGYA_ROUTER_H
#define PRAGYA_ROUTER_H

#include "../include/pragya_avastha.h"

void pragya_route(PragyaAvastha *avastha);

#endif

```

```

=====
FILE: ./pragya/niyu/niyu_contract.h
=====
#ifndef NIYU_CONTRACT_H
#define NIYU_CONTRACT_H

#include "../include/pragya_avastha.h"

/* ===== */
/* NIYU ERROR TYPES      */
/* ===== */

typedef enum {

    NIYU_ERR_VALID = 0,
    NIYU_ERR_SYNTAX,
    NIYU_ERR_MISSING_OPERAND,
    NIYU_ERR_EXTRA_OPERATOR,
    NIYU_ERR_INVALID_IDENTIFIER,
    NIYU_ERR_UNDEFINED_VARIABLE,
    NIYU_ERR_TYPE_MISMATCH,
    NIYU_ERR_DIV_ZERO,
    NIYU_ERR_UNEXPECTED_TOKEN,
    NIYU_ERR_EMPTY_INPUT,

```



```

        NIYU_ERR_INVALID_COMMAND,
        NIYU_ERR_RUNTIME

} NiyuErrorType;

/* ===== */
/* NIYU INTENT TYPES */
/* ===== */

typedef enum {

    NIYU_INTENT_CALCULATION = 0,
    NIYU_INTENT_DECLARATION,
    NIYU_INTENT_PRINT,
    NIYU_INTENT_UNKNOWN

} NiyuIntent;

/* ===== */
/* NIYU RESULT STRUCT */
/* ===== */

typedef struct {

    NiyuIntent intent;
    NiyuErrorType error_type;

    int severity;

    int error_pos;

} NiyuResult;

/* ===== */
/* NIYU CORE API */
/* ===== */

NiyuResult* niyu_think_avastha(PragyaAvastha *avastha);

void niyu_free_result(NiyuResult *res);

#endif

=====
FILE: ./pragya/niyu/niyu_core.c
=====
#include "niyu_contract.h"
#include "../include/pragya_avastha.h"

#include <stdlib.h>
#include <string.h>
#include <ctype.h>

/* ===== */
/* Helper functions */
/* ===== */

static int is_operator(char c)
{
    return (c=='+' || c=='-' || c=='*' || c=='/');
}

static int is_digit_char(char c)
{
    return isdigit((unsigned char)c);
}

```

```

}

static int starts_with(const char *text, const char *word)
{
    return strncmp(text, word, strlen(word)) == 0;
}

/* ===== */
/* Skip spaces */
/* ===== */

static int next_non_space(const char *t, int i)
{
    while (t[i] && isspace((unsigned char)t[i]))
        i++;

    return i;
}

/* ===== */
/* NIYU THINK ENGINE */
/* ===== */

NiyuResult* niyu_think_avastha(PragyaAvastha *a)
{
    if (!a || !a->raw_text)
        return NULL;

    const char *t = a->raw_text;

    NiyuResult *res = malloc(sizeof(NiyuResult));

    if (!res)
        return NULL;

    res->intent = NIYU_INTENT_UNKNOWN;
    res->error_type = NIYU_ERR_VALID;
    res->severity = 0;
    res->error_pos = -1;

    int len = strlen(t);

    if (len == 0)
    {
        res->error_type = NIYU_ERR_EMPTY_INPUT;
        res->severity = 1;
        return res;
    }

    /* ===== */
    /* INTENT DETECTION */
    /* ===== */

    if (starts_with(t, "mavi"))
    {
        res->intent = NIYU_INTENT_DECLARATION;
        return res;
    }

    if (starts_with(t, "bolo"))
    {
        res->intent = NIYU_INTENT_PRINT;
        return res;
    }

    res->intent = NIYU_INTENT_CALCULATION;

    /* ===== */

```

```

/* ERROR PATTERN DETECTION */
/* ===== */

for (int i = 0; i < len; i++)
{
    int j = next_non_space(t, i+1);

    if (!t[j])
        break;

    char c1 = t[i];
    char c2 = t[j];

    /* operator operator */

    if (is_operator(c1) && is_operator(c2))
    {
        res->error_type = NIYU_ERR_EXTRA_OPERATOR;
        res->severity = 2;
        res->error_pos = j;
        return res;
    }

    /* digit digit (missing operator) */

    if (is_digit_char(c1) && is_digit_char(c2))
    {
        if (j > i+1)
        {
            res->error_type = NIYU_ERR_SYNTAX;
            res->severity = 2;
            res->error_pos = j;
            return res;
        }
    }
}

/* ===== */
/* Missing operand */
/* ===== */

int last = len-1;

while (last >= 0 && isspace((unsigned char)t[last]))
    last--;

if (last >= 0 && is_operator(t[last]))
{
    res->error_type = NIYU_ERR_MISSING_OPERAND;
    res->severity = 2;
    res->error_pos = last;
    return res;
}

/* ===== */
/* Undefined variable heuristic */
/* ===== */

if (isalpha((unsigned char)t[0]))
{
    res->error_type = NIYU_ERR_UNDEFINED_VARIABLE;
    res->severity = 3;
    res->error_pos = 0;
    return res;
}

return res;
}

```

```

/* ===== */
/* FREE RESULT */
/* ===== */

void niyu_free_result(NiyuResult *res)
{
    if (res)
        free(res);
}

```

```

=====
FILE: ./pragya/niyu/niyu_core.h
=====
#ifndef NIYU_CORE_H
#define NIYU_CORE_H

#include "niyu_contract.h"

/*
 * NIYU CORE
 *
 * Implementation file will:
 * - define NiyuResult structure
 * - implement internal logic
 *
 * This header exposes NOTHING extra.
 * Future machine-backed engine can replace niyu_core.c
 * without touching any other file.
 */

#endif /* NIYU_CORE_H */

```

```

=====
FILE: ./include/user_config.h
=====
#ifndef USER_CONFIG_H
#define USER_CONFIG_H

// Personality mapping
#define P_SAKHI 1
#define P_NIYU 2
#define P_MIRA 3
#define P_KAVYA 4
#define P_SHIRI 5
#define P_ALL 6 // Unified AI

// Mode mapping
#define MODE_INDIA 1 // Hindi-English mix
#define MODE_GLOBAL 2 // Full English

// Developer mode flag (ON by default)
extern int DEV_MODE;

// Current personality (default = Sakhi)
extern int CURRENT_PERSONALITY;

// Current mode (default = INDIA)
extern int CURRENT_MODE;

void config_init();
void config_set_personality(int p);
void config_set_mode(int mode);

#endif

```


[illegible]

```

=====
FILE: ./include/fix_engine.h
=====
/* RADHE_RADHE_SHREEJI */

/* =====
   SHRIJILANG â\200\224 EXECUTION ENGINE (CORE CONTRACT)
   ===== */

#ifndef SHRIJI_ENGINE_H
#define SHRIJI_ENGINE_H

#include "ast.h"
#include "value.h"
#include "env.h"
#include "runtime.h"
#include "error.h"

/* =====
   ENGINE CONFIG
   ===== */

/* Maximum number of fix attempts allowed */
#define ENGINE_MAX_FIX_ATTEMPTS 5

/* =====
   ENGINE STATUS
   ===== */

typedef enum
{
    ENGINE_OK = 0,
    ENGINE_PARSE_ERROR,
    ENGINE_RUNTIME_ERROR,
    ENGINE_FIX_FAILED
} EngineStatus;

/* =====
   ENGINE RESULT STRUCTURE
   ===== */

typedef struct
{
    /* ----- Language Layer ----- */

    ASTNode *ast;          /* Final AST (after fix if any) */

    /* ----- Runtime Layer ----- */

    Value result;          /* Final evaluated result */

    /* ----- Fix Engine Meta ----- */

    int was_fixed;          /* 1 if any fix applied */
    int fix_attempts;       /* number of attempts used */
    int confidence_penalty; /* total penalty from fixes */

    /* ----- Status ----- */

    EngineStatus status;    /* final execution status */

} EngineResult;

/* =====
   ENGINE ENTRY POINT
   ===== */

/*
   Main execution pipeline:

```

```

Input
    â\206\222 Parse
    â\206\222 Fix (if needed)
    â\206\222 Re-parse
    â\206\222 Runtime
    â\206\222 Result

NOTE:
- This function is PURE (no printing ideally in future)
- KRST should control how it is used
*/
EngineResult shriji_engine_execute(
    const char *input,
    Env *env
);

/* =====
LOW-LEVEL CONTROL (INTERNAL USE)
===== */

/* Single parse attempt (no fix) */
ASTNode *engine_parse_once(const char *input);

/* Apply fix rules */
int engine_apply_fix(
    char *buffer,
    size_t size,
    const ShrijiErrorInfo *err
);

/* =====
SAFETY UTILITIES
===== */

/* Reset result safely */
void engine_result_init(EngineResult *res);

/* Free result safely */
void engine_result_free(EngineResult *res);

#endif /* SHRIJI_ENGINE_H */

=====
FILE: ./include/typo_engine.h
=====
#ifndef SHRIJI_TYPO_ENGINE_H
#define SHRIJI_TYPO_ENGINE_H

/*
    Phase-1 Typo Suggestion Engine
    Only checks for "exit" typo
*/

int shriji_check_typo(
    const char *input,
    char *suggestion,
    int suggestion_size
);

#endif

=====
FILE: ./include/smriti_personal.h
=====
#ifndef SMRITI_PERSONAL_H
#define SMRITI_PERSONAL_H

```


PERSONAL SMRITI API

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```

typedef enum {
    AI_INTENT_UNKNOWN = 0,
    AI_INTENT_EXPLAIN,    /* samjhao, explain, batao */
    AI_INTENT_WHY,        /* why, kyun, kyu */
    AI_INTENT_CALC,       /* calculate, solve, math */
    AI_INTENT_EMOTION     /* sad, dukhi, lonely */
} AIIntent;

/* raw text -> detected intent */
AIIntent ai_detect_intent(const char *message);

#endif /* AI_INTENT_H */

=====
FILE: ./include/decision_engine.h
=====
#ifndef DECISION_ENGINE_H
#define DECISION_ENGINE_H

#include "error.h"

typedef enum {
    DECISION_AUTO_FIX,
    DECISION_EDIT,
    DECISION_REJECT
} DecisionType;

/* classification */
int is_safe_error(ShrijiErrorCode code);
int is_ambiguous_error(ShrijiErrorCode code);

/* main decision */
DecisionType shriji_take_decision(
    int confidence,
    int risk,
    ShrijiErrorCode code
);

#endif

=====
FILE: ./include/lang/lang_config.h
=====
#ifndef SHRIJI_LANG_CONFIG_H
#define SHRIJI_LANG_CONFIG_H

#define SHRIJI_ENABLE_MASTER_TOKENS

#endif

=====
FILE: ./include/lang/token_master.h
=====
#ifndef SHRIJI_TOKEN_MASTER_H
#define SHRIJI_TOKEN_MASTER_H

/*
    SHRIJILANG â\200\224 MASTER TOKEN REGISTRY
    -----
    This file is a DESIGN + FUTURE registry.

    RULES:
    1. token.h is the runtime truth
    2. Only tokens present in token.h are ACTIVE
    3. Others are FUTURE / PARKED (no runtime meaning yet)

```

```

*/

/* =====
CORE / ACTIVE TOKENS (MIRROR OF token.h)
===== */

typedef enum {

    /* ----- Special ----- */
    MT_TOKEN_EOF = 0,
    MT_TOKEN_ERROR,
    MT_TOKEN_NEWLINE,

    /* ----- Literals ----- */
    MT_TOKEN_NUMBER,
    MT_TOKEN_IDENTIFIER,
    MT_TOKEN_STRING,
    MT_TOKEN_TRUE,
    MT_TOKEN_FALSE,

    /* ----- Core Keywords ----- */
    MT_TOKEN_MAVI,
    MT_TOKEN_AGAR,
    MT_TOKEN_WARNNA,
    MT_TOKEN_KAAM,
    MT_TOKEN_JABTAK,
    MT_TOKEN_RUKJA,
    MT_TOKEN_CHALU,
    MT_TOKEN_NAHI,
    MT_TOKEN_BOLO,
    MT_TOKEN_WAPAS,
    MT_TOKEN_IMPORT,

    /* ----- AI / System Commands ----- */
    MT_TOKEN_SAKHI,
    MT_TOKEN_NIYU,
    MT_TOKEN_MIRA,
    MT_TOKEN_KAVYA,
    MT_TOKEN_SHIRI,
    MT_TOKEN_SAMAY,

    /* ----- Operators ----- */
    MT_TOKEN_PLUS,
    MT_TOKEN_MINUS,
    MT_TOKEN_STAR,
    MT_TOKEN_SLASH,
    MT_TOKEN_MOD,

    MT_TOKEN_EQUAL,
    MT_TOKEN_EQEQ,
    MT_TOKEN_NEQ,
    MT_TOKEN_GT,
    MT_TOKEN_LT,
    MT_TOKEN_GTE,
    MT_TOKEN_LTE,

    /* ----- Increment / Decrement ----- */
    MT_TOKEN_PLUSPLUS,
    MT_TOKEN_MINUSMINUS,

    /* ----- Logical ----- */
    MT_TOKEN_AND,
    MT_TOKEN_OR,
    MT_TOKEN_NOT,

    /* ----- Punctuation ----- */
    MT_TOKEN_LEFT_PAREN,
    MT_TOKEN_RIGHT_PAREN,
    MT_TOKEN_LEFT_BRACE,

```



```

MT_TOKEN_RIGHT_BRACE,
MT_TOKEN_LEFT_BRACKET,
MT_TOKEN_RIGHT_BRACKET,
MT_TOKEN_COMMA,
MT_TOKEN_COLON,

/* =====
FUTURE / PARKED TOKENS (NOT IN token.h YET)
===== */

/* ----- Declarations ----- */
MT_TOKEN_CONST,
MT_TOKEN_LET,

/* ----- Types ----- */
MT_TOKEN_TYPE_INT,
MT_TOKEN_TYPE_FLOAT,
MT_TOKEN_TYPE_STRING,
MT_TOKEN_TYPE_BOOL,
MT_TOKEN_TYPE_LIST,
MT_TOKEN_TYPE_DICT,
MT_TOKEN_TYPE_ANY,

/* ----- Control Flow (future) ----- */
MT_TOKEN_FOR,
MT_TOKEN_DO,
MT_TOKEN_SWITCH,
MT_TOKEN_CASE,
MT_TOKEN_DEFAULT,

/* ----- Assignment Variants ----- */
MT_TOKEN_PLUSEQ,
MT_TOKEN_MINUSEQ,
MT_TOKEN_STAREQ,
MT_TOKEN_SLASHEQ,
MT_TOKEN_MODEQ,

/* ----- Strict / Advanced Compare ----- */
MT_TOKEN_STRICT_EQ,
MT_TOKEN_STRICT_NEQ,

/* ----- Bitwise ----- */
MT_TOKEN_BIT_AND,
MT_TOKEN_BIT_OR,
MT_TOKEN_BIT_XOR,
MT_TOKEN_BIT_NOT,
MT_TOKEN_SHIFT_LEFT,
MT_TOKEN_SHIFT_RIGHT,

/* ----- Ternary ----- */
MT_TOKEN_QUESTION,

/* ----- Structure (future sugar) ----- */
MT_TOKEN_DOT,
MT_TOKEN_ARROW,

/* ----- Data Structures ----- */
MT_TOKEN_LIST,
MT_TOKEN_DICT,
MT_TOKEN_SET,
MT_TOKEN_TUPLE,

/* ----- Sentinel ----- */
MT_TOKEN__COUNT

} ShrijiMasterToken;

#endif /* SHRIJI_TOKEN_MASTER_H */

```



```

static inline Value value_number(double n)
{
    Value v = value_null();
    v.type = VAL_NUMBER;
    v.number = n;
    return v;
}

static inline Value value_string(const char *s)
{
    Value v = value_null();
    v.type = VAL_STRING;

    if (!s) {
        v.string = NULL;
        return v;
    }

    v.string = (char *)malloc(strlen(s) + 1);
    if (!v.string) return value_null();

    strcpy(v.string, s);
    return v;
}

/* BOOL SUPPORT */
static inline Value value_bool(int b)
{
    Value v = value_null();
    v.type = VAL_BOOL;
    v.boolean = b ? 1 : 0;
    return v;
}

static inline int value_is_truthy(Value v)
{
    switch (v.type) {
        case VAL_BOOL:    return v.boolean;
        case VAL_NUMBER:  return v.number != 0;
        case VAL_STRING:  return v.string && v.string[0] != '\0';
        case VAL_NULL:    return 0;
        default:          return 1; // list, dict, function â\206\222 truthy
    }
}

/* â\234\205 list constructor */
static inline Value value_list(Value *items, int count)
{
    Value v = value_null();
    v.type = VAL_LIST;
    v.items = items;
    v.count = count;
    return v;
}

/* â\234\205 function constructor */
static inline Value value_function(ASTNode *fn)
{
    Value v = value_null();
    v.type = VAL_FUNCTION;
    v.function = fn;
    return v;
}

/* â\234\205 dict constructor */
static inline Value value_dict(Value *keys, Value *values, int count)
{
    Value v = value_null();

```



```

=====
FILE: ./include/translator.h
=====
#ifndef TRANSLATOR_H
#define TRANSLATOR_H

// Translate English/world command â\206\222 ShrijiLang keyword
const char* translate_keyword(const char *word);

// Autocorrect system for mistyped user commands
const char* mira_autocorrect(const char *word);

// Extract first word from input (helper for parser)
char* get_first_word(const char *source);

#endif

```

```

=====
FILE: ./include/nirman_flow_engine.h
=====
#ifndef NIRMAN_FLOW_ENGINE_H
#define NIRMAN_FLOW_ENGINE_H

void nirman_flow_process(char *line);

#endif

```

```

=====
FILE: ./include/nirman.h
=====
#ifndef NIRMAN_H
#define NIRMAN_H

/* =====
CONTEXT STRUCT
===== */
typedef struct {
    int active;           /* nirman mode ON/OFF */
    int flow_stage;       /* state machine stage */
    char goal[256];       /* user goal */
} NirmanContext;

/* =====
GLOBAL ACCESS
===== */
extern NirmanContext NIRMAN_CTX;

/* =====
LIFECYCLE
===== */
void nirman_start(void);
void nirman_stop(void);
int nirman_is_active(void);

/* =====
INTENT ENGINE
===== */
int nirman_detect_intent(const char *input);

#endif

```

```

=====
FILE: ./include/pragya_avastha.h
=====
#ifndef PRAGYA_AVASTHA_H
#define PRAGYA_AVASTHA_H

```

```

#include "ast.h"
#include "error.h"
#include "value.h"

/*
    PragyaAvastha

    Shriji system ki vartamaan sthiti.
    Yeh packet KRST se Pragya tak jata hai
    aur fir Niyu analysis ke liye use karti hai.
*/

typedef struct {

    /* User input */
    const char *raw_text;

    /* NEW: input mode */
    int is_program_mode;

    /* NEW: correction system */
    int has_correction;
    char corrected_text[256];
    int stop_execution;

    /* Parsed structure */
    ASTNode *ast;

    /* Error state */
    ShrijiErrorCode error_code;

    int error_pos;
    int error_len;

    /* KRST metrics */
    int confidence;
    int risk;

    int teach_level;
    int tone;
    int allow_auto_fix; /* KRST will decide fix permission */
    /* Runtime result */
    Value runtime_result;

} PragyaAvastha;

#endif

=====
FILE: ./include/example_builder.h
=====
#ifndef SHRIJI_EXAMPLE_BUILDER_H
#define SHRIJI_EXAMPLE_BUILDER_H

void shriji_build_example(
    const char *input,
    char *out,
    int size
);

#endif

=====
FILE: ./include/normalize_engine.h
=====
#ifndef NORMALIZE_ENGINE_H

```


[illegible]

[illegible]


```

    TOKEN_WAPAS,

    TOKEN_IMPORT,
    TOKEN_RACHNA,

    /* Operators */
    TOKEN_PLUS,
    TOKEN_MINUS,
    TOKEN_STAR,
    TOKEN_SLASH,
    TOKEN_MOD,

    TOKEN_EQUAL,
    TOKEN_EQEQ,
    TOKEN_NEQ,
    TOKEN_GT,
    TOKEN_LT,
    TOKEN_GTE,
    TOKEN_LTE,

    /* Increment / Decrement */
    TOKEN_PLUSPLUS,    // ++
    TOKEN_MINUSMINUS,  // --

    /* Logical Operators */
    TOKEN_AND,          // &&
    TOKEN_OR,           // ||
    TOKEN_NOT,          // !

    /* Punctuation */
    TOKEN_LEFT_PAREN,
    TOKEN_RIGHT_PAREN,
    TOKEN_LEFT_BRACKET,
    TOKEN_RIGHT_BRACKET,
    TOKEN_LEFT_BRACE,
    TOKEN_RIGHT_BRACE,
    TOKEN_COMMA,
    TOKEN_COLON,
    TOKEN_SEMICOLON,
    TOKEN_NEWLINE,

    /* Special */
    TOKEN_ERROR
} TokenType;

typedef struct {
    TokenType type;
    const char *start;
    int length;
    int line;
    int col;
} Token;

/* Scanner API */
void init_tokenizer(const char *source);
Token scan_token(void);

/* #ifdef SHRIJI_ENABLE_MASTER_TOKENS
/* #include "lang/token_master.h"
/* #endif

#endif // TOKEN_H

=====
FILE: ./include/fix_interactive.h
=====
#ifndef SHRIJI_FIX_INTERACTIVE_H

```

```
#include <stddef.h>
#include "error.h"
```

```
int shriji_interactive_operator_fix(
    const char *input,
    char *output,
    int size
);
```

```
int shriji_interactive_bracket_fix(
    const char *input,
    char *output,
    int size
);
```

```
int shrij_i_interactive_double_operator_fix(
    const char *input,
    char *output,
    int size
);
```

```
=====
FILE: ./include/mira_engine.h
=====
```

```
void mira_teach(const char *msg);

#endif
```

```
=====
FILE: ./include/interpreter.h
=====
```

```
#include "env.h"
#include "ast.h"
#include "value.h"
#include "runtime.h" // 237\224 NEW 200\224 runtime struct access
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

